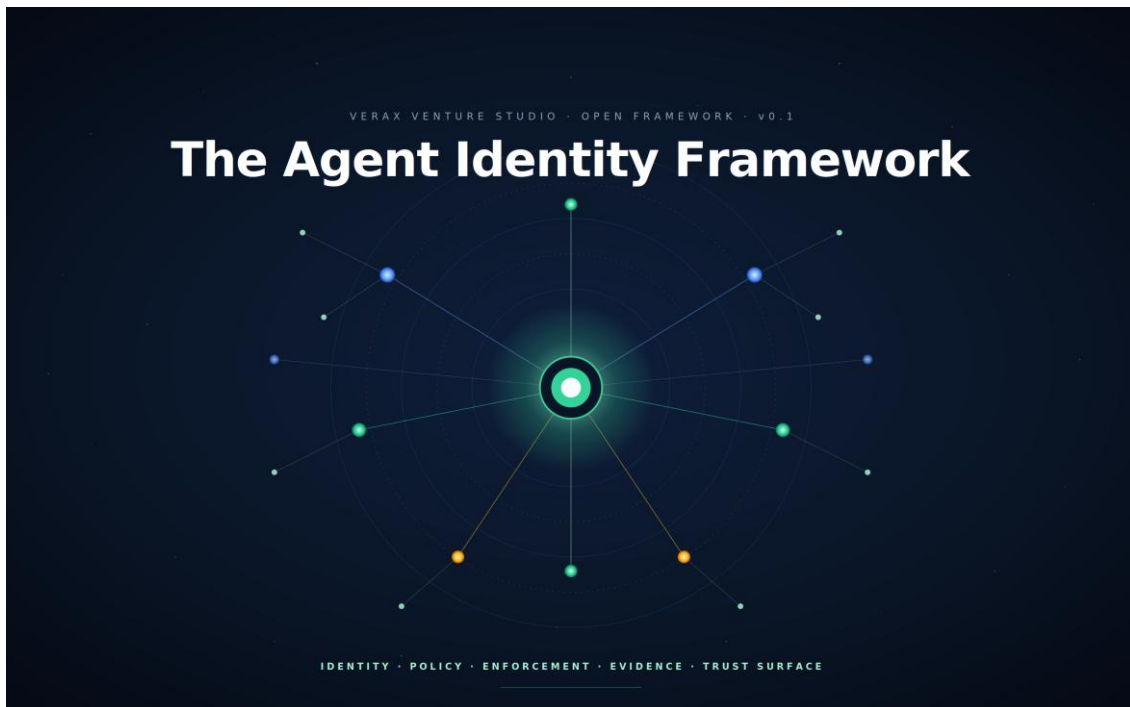




Authored by

Anthony Penwright

Version

0.1 , Public Draft for Review (corrected)

Date

April 2026

Licence (intent)

CC-BY-SA 4.0 (specification) · Apache 2.0 (reference implementations)

Status

Open framework , contributions invited

Copyright

© 2026 Anthony Penwright. All rights reserved in the trademarks and trade dress; specification text licensed under CC-BY-SA 4.0.

This document is offered openly to operators, regulators, standards bodies, insurers and customers of agentic AI systems. It will be useful to the extent it is adopted, contributed to and improved by the community.

A stylized, handwritten signature in black ink, consisting of a large 'A' followed by a horizontal line and a diagonal stroke.

2026-07-20

Anthony

2026-07-20

Notice , Copyright, Trademarks and Attribution

Copyright and licence

This document, including all text, figures, tables and the architectural framework it describes, is © 2026 Anthony Penwright and Verax Venture Studio. The specification, that is, the prose, diagrams and conceptual framework set out in this paper, is published under the Creative Commons Attribution-ShareAlike 4.0 International Licence (CC-BY-SA 4.0). The full licence text is available at <https://creativecommons.org/licenses/by-sa/4.0/>. Reference implementations, where published separately, are released under the Apache Licence 2.0.

In short: you are free to share, adapt and build upon this work for any purpose, including commercially, provided that you (a) give appropriate credit to Anthony Penwright and Verax Venture Studio as the original authors, (b) provide a link to this licence, (c) indicate if changes were made, and (d) distribute any derivative works of the specification under the same CC-BY-SA 4.0 terms. The ShareAlike requirement ensures that improvements to the framework remain open to the community.

Trademarks

The licence above covers the specification text only.

Implementations of the framework are welcome and indeed actively encouraged. A separate framework-conformance mark, governed by a community-defined test suite, is in development; this paper will be updated when that mark becomes available for self-certification or third-party attestation.

Attribution requirements for derivative works

Where this work is quoted, summarised, adapted or built upon, attribution should take a form substantially as follows: "Based on The Agent Identity Framework v0.1, by Anthony Penwright (April 2026), licensed under CC-BY-SA 4.0." Where the derivative work is itself a written specification or published document, the attribution should appear in a location at least as prominent as a copyright notice would conventionally appear in such a work. Where the derivative is a software implementation, attribution in the project README and in any public-facing documentation is sufficient. Verbatim reproduction of substantial passages, beyond the limits of fair dealing or fair use under applicable law, requires the same attribution and the same ShareAlike onward licensing.

First publication and prior art

This document was first published on 27 April 2026. A SHA-256 cryptographic hash of the published file has been generated and timestamped via OpenTimestamps, which anchors the hash to the Bitcoin blockchain as a tamper-evident record of the document's existence on that date. The framework,

including all techniques, constructs and architectural patterns described in this paper, is offered as a public defensive disclosure: by virtue of this publication, the material constitutes prior art and may not be the subject of subsequently filed patents.

Contact

Questions, contributions, conformance enquiries and trademark licensing requests should be directed to ap@credible.uk.com.

Contents

1. Premise, Why agents need identity	10
1.1 The three operating modes	11
1.2 The five-layer model	12
2. Identity Lifecycle	14
2.1 Birth, provisioning	14
2.2 Profile	15
2.3 Operate	15
2.4 Evolve	15
2.5 Retire	15
3. Master System of Record , the Agent Identity Provider	17
3.1 The core schema	17
3.2 The Agent Bill of Materials	17
3.3 Lineage as a first-class graph	18
3.4 Federation	18
4. Profile, Skills and Blessings	20
4.1 The three components	20
4.2 Skills as versioned, signed artefacts	20
4.3 Blessings, earned credentials	20
4.4 Memory hygiene	21
5. Fine-Grained Permissioning	22
5.1 Capabilities, not roles	22
5.2 Scope-bound tokens	22
5.3 Just-in-time elevation	22
5.4 Consent receipts	23
5.5 Human-in-loop gates	23
6. The Policy Enforcement Plane	24
6.1 What the enforcement plane enforces	24
6.2 Trace propagation for recursive and multi-agent calls	25
6.3 Audit emission as a first-class output	25
6.4 FinOps as a control surface	25
6.5 Multi-tenant policy partition	26
6.6 Reference stacks	26
7. The Audit Layer	28
7.1 What gets logged	28
7.2 Tamper evidence and admissibility	28

7.3 Replay.....	29
7.4 Storage tiers	29
7.5 What never goes in the audit log.....	30
8. Self-Learning Provenance	31
8.1 What counts as learning	31
8.2 The provenance chain.....	31
8.3 Preventing capability drift.....	31
8.4 Preventing compliance laundering via fine-tuning	32
9. Visualisation and Operator Surfaces.....	33
9.1 The four canonical views.....	33
9.2 The data pipeline.....	33
9.3 Operator workflows	34
9.4 Public-facing transparency surfaces	34
10. Data Compliance, Sovereignty and Regulatory Alignment	35
10.1 Data classification at the gateway	35
10.2 Lawful basis and purpose limitation	35
10.3 Data residency and sovereignty.....	35
10.4 The retention–erasure paradox and crypto-shredding	36
10.5 EU AI Act alignment	36
10.6 Sector overlays.....	37
10.7 DPIA and Algorithmic Impact Assessment per agent class	37
10.8 Compliance as a product surface.....	37
11. Risk, Ethics, Guardrails and Litigation Defence.....	39
11.1 Risk taxonomy	39
11.2 Defence in depth, six layers of guardrail.....	39
11.3 Rulebooks, codified, versioned, attestable.....	40
11.4 Ethics governance, three tiers	41
11.5 Behavioural defence in litigation	41
11.6 Continuous red-teaming	42
11.7 Liability allocation	43
11.8 Insurance and risk transfer	43
11.9 Incident response and forensics	43
11.10 Practicalities and realities, the maturity ladder.....	43
12. Multi-tenancy, Federation and Cross-Organisation Agent-to-Agent Operation	46
12.1 Multi-tenancy as a cross-cutting concern.....	46
12.2 Federation between trust roots.....	46
12.3 Agent-to-Agent protocols	47
12.4 Cross-organisation policy enforcement plane peering.....	48

12.5 Standards and conformance map	48
13. Reference Architecture, Developer Experience and Golden Paths	50
13.1 Reference architecture	50
13.2 Developer experience , the golden path.....	51
13.3 Adoption staging , incremental, not big-bang	52
14. Framework Governance, Open Questions and Call to Action	53
14.1 Why this needs to be open	53
14.2 Governance model.....	53
14.3 Open questions	53
14.4 Call to action	54
Appendix A, Glossary	55
Appendix B, Standards Referenced	56
Appendix C, Versioning.....	57

Abstract

Identity has always been the foundation of every other control in software. Users, services, machines and equipment are each enrolled, attested, scoped, audited and retired. Each carries a credential, a record, a history. Autonomous and semi-autonomous AI agents, software actors driven by large language models, capable of reasoning, calling tools, spawning sub-agents and learning over time, break the assumptions on which traditional Identity and Access Management was built. Sessions span minutes to days rather than seconds. Scopes expand recursively rather than holding static. Actions are probabilistic rather than deterministic, and the principal chain is multi-hop rather than singular. A single agent may operate across many delegations, many tenants and many jurisdictions in the course of a single working day.

This whitepaper proposes the Agent Identity Framework: an open, layered specification that brings agents to identity parity with humans, services and machines, and that produces, as a first-class output rather than as a forensic afterthought, the evidence that regulators, customers, insurers and courts will increasingly require. The framework decomposes into five concentric layers: Identity (the master system of record and the agent identity provider), Policy (capabilities, scopes and lawful basis), Enforcement (a non-bypassable gateway plane), Evidence (a tamper-evident, hash-chained audit log with deterministic replay), and Trust Surface (operator and customer-facing transparency). Each layer is independently valuable and sequentially adoptable; together they constitute the difference between an organisation that has large language models in production and one that has governed agents in production.

The framework treats every agent action as falling into one of three modes, human-led, human-intervened or autonomous, and records that mode per action rather than per agent. It treats learning as a credentialled event rather than a silent drift. It treats fine-tuning as the birth of a new agent rather than an in-place update. It treats compliance not as a bolt-on but as the property that emerges when identity, policy enforcement, audit and visualisation are correctly co-designed. It treats federation as mandatory: an agent that cannot present its identity, capabilities and ethics review outside its home organisation is not a citizen of the agent economy but a cul-de-sac.

This document is offered openly. The specification is licensed under CC-BY-SA 4.0; reference implementations are licensed under Apache 2.0. Stewardship rests with Verax Venture Studio for version 0.x, with a defined path to a neutral foundation once adoption justifies it. The framework is intended to be useful to the extent it is adopted, contributed to and improved by operators, standards bodies, regulators, insurers and customers, and the call to action at the end of this paper is addressed to each of those audiences in turn.

Executive Summary

The thesis. Agents are first-class actors. They must be enrolled, attested, scoped, audited and retired with the same rigour applied to humans, services and machines, and then more, because their scope is generative, their sessions are long, their authority chains are multi-hop, and their actions are probabilistic. Without identity, all other controls are advisory. Without enforcement, identity is decorative. Without evidence, enforcement is unprovable. Without a trust surface, evidence is forensic-only. The five layers must be co-designed; none of them is sufficient on its own.

What is new. Existing IAM, API management and observability stacks each address part of the problem. None addresses the full set. The Agent Identity Framework integrates the Agent Identity Provider, the Agent Bill of Materials, capability-based authorisation with just-in-time elevation, the policy enforcement plane, the hash-chained audit log with deterministic replay, the four canonical visualisation surfaces, and the rulebook construct that turns ethical principles into testable assertions bound to enforcement layers and evidence events. It is, deliberately, an integration, and the integration is the contribution.

Who this is for. Operators running agents in production today. Standards bodies extending existing identity, credential and audit specifications to autonomous actors. Regulators framing supervisory expectations under the EU AI Act, GDPR, sectoral rules and emerging national regimes. Insurers and underwriters binding cover to AI liability. Customers procuring agents that touch their data. Engineers and product leaders authoring the next generation of agent-native systems.

How to read it. The paper proceeds in fourteen sections. Section 1 establishes the premise and the five-layer model. Section 2 describes the agent lifecycle. Sections 3 to 5 set out the identity, profile and permissioning primitives. Section 6 covers the enforcement plane. Section 7 covers evidence. Section 8 addresses self-learning provenance. Section 9 covers visualisation. Section 10 addresses data compliance and sovereignty. Section 11 addresses risk, ethics, guardrails and litigation defence. Section 12 covers multi-tenancy, federation and agent-to-agent operation across organisational boundaries. Section 13 sets out a reference architecture and developer experience. Section 14 covers governance, open questions and a call to action.

What to do on Monday morning. Pick one section. Adopt it. Identity first if you have nothing; the policy enforcement plane second if you already have identity; audit third. The maturity ladder in section 11 sets out the staged path; total adoption time for a small team to reach a defensible baseline is approximately one quarter, parallelised. The framework is designed to make stage four cheaper than stage one incident response, and that is the case it makes to the executive sponsor.

1. Premise, Why agents need identity

Software has always had identity for the things that act. Users are issued credentials, granted roles, monitored and retired. Services are issued certificates, scoped to APIs and rate-limited. Machines are enrolled, attested at boot, and given workload identities. Robots and process equipment in product-lifecycle management systems carry serial numbers, calibration records and operating envelopes. In every case the pattern is the same: a stable identifier, a credential, a record of authority, a record of behaviour, and a defined end of life. The pattern works because it makes the actor accountable, discoverable in an audit, revocable on incident, comparable across time.

Autonomous and semi-autonomous AI agents fit none of the assumptions on which the existing pattern was built. The differences are not incremental. They are categorical, and they invalidate, item by item, the architectural shortcuts that made traditional Identity and Access Management efficient. A user is human; an agent is generative, an agent can spawn a sub-agent on its own initiative, and that sub-agent must itself have an identity. A user has a session that lasts seconds; an agent may operate continuously over days, weeks or months. A user's scope is static; an agent's scope expands recursively as it acquires tools, learns skills and earns credentials. A user's actions are deterministic and traceable to a single principal; an agent's actions are probabilistic, the principal chain is multi-hop, and the question "on whose behalf was this done" is genuinely hard. A user's audit log is forensic, consulted after incidents; an agent's audit log is evidentiary, regulatory and operational, consulted continuously.

The table below contrasts the assumptions of legacy IAM with the realities of agent operation. It is offered not as a critique of legacy systems, they remain correct for their original domains, but as a precise enumeration of the gaps the Agent Identity Framework is designed to close.

Assumption (legacy IAM)	Reality (agents)
Identity is human or machine	Identity is generative, it spawns sub-identities
Sessions are short, scopes are static	Sessions span minutes to days; scope expands recursively
Actions are deterministic and traceable to a single principal	Actions are probabilistic; the principal chain is multi-hop
Authorisation = "is this person allowed?"	Authorisation = "is this agent, acting for whom, with what learned capabilities, under what lawful basis, allowed?"
Audit logs are forensic	Audit logs are evidentiary, regulatory and operational
One identity, one human	One agent, many delegations, many tenants, many jurisdictions

The gap between "we have a large language model in production" and "we have a governed agent in production" is the gap this framework closes. The premise is therefore plain. An agent is a software

actor with a stable identity, a declared purpose, a bounded set of capabilities, the ability to call tools and other agents, and a recorded behavioural history. An agent is first-class: it carries identity parity with humans, services and machines, and it is treated by every adjacent system, policy engines, audit logs, visualisation, compliance, ethics review, on those terms.

Working definition

An agent is a software actor with a stable identity, a declared purpose, a bounded set of capabilities, the ability to call tools and other agents, and a recorded behavioural history. An agent is first-class, it carries identity parity with humans, services and machines, and the framework treats it accordingly throughout.

1.1 The three operating modes

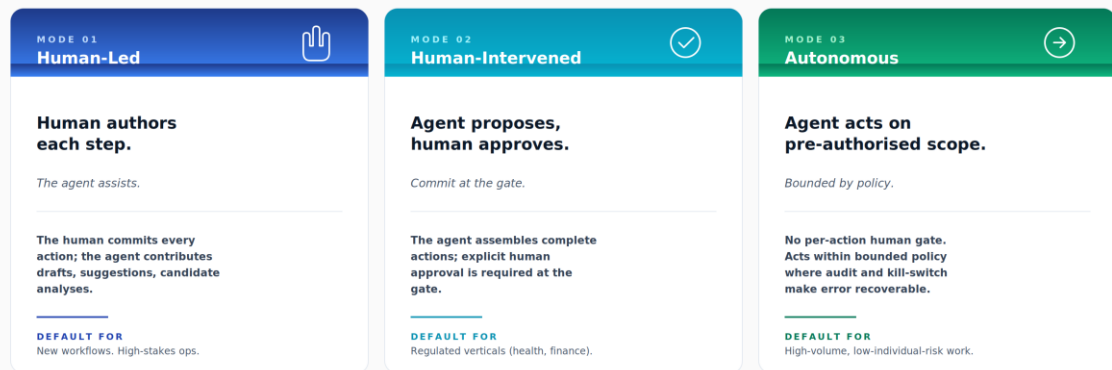
Every agent action falls into one of three modes, and the framework requires all three to be treated as legitimate, evidenced and distinguishable in audit. The first mode is human-led: the human authors each step and the agent assists, contributing drafts, retrievals, summaries or candidate actions for the human to commit. This is the default mode for new workflows that have not yet earned the trust required for higher autonomy, and for high-stakes operations where the cost of error is significant. The second mode is human-intervened: the agent proposes a course of action and the human approves it before commitment. This is the default mode for regulated verticals, healthcare, financial services, public sector, where supervisory expectations require demonstrable human oversight. The third mode is autonomous: the agent acts within a pre-authorised scope without per-action human approval. This is the appropriate mode for high-volume, low-risk operations whose risk profile has been characterised, whose guardrails are mature, and whose tolerance for occasional deviation is well understood.

Mode is recorded per action, not per agent. The same agent will operate in all three modes across a single working session: human-led when drafting an unfamiliar deliverable, human-intervened when sending an external communication, autonomous when enriching an internal record. This per-action discipline matters because audit defensibility, regulatory mapping and incident triage all depend on knowing, at the moment a specific action was taken, which mode the agent was in and which authority underwrote the action. A framework that classifies mode at the level of the agent rather than the action has already discarded the resolution it most needs.

FIGURE 2 · THE THREE OPERATING MODES

Mode is per-action, not per-agent.

A single agent moves between all three modes within a session — and the audit log records mode for every event.



The audit log records mode for every event — mode determines liability, defensibility, and post-hoc forensic standard.

1.2 The five-layer model

The framework decomposes into five layers, each with a distinct purpose, each independently valuable, and each sequentially adoptable. The layers are arranged conceptually as a stack from foundation to surface, but they must be co-designed: skipping any layer reduces every layer above it to advisory status.

Layer one is Identity. It comprises the master system of record, the agent identity provider, the profiles, capabilities, blessings and lineage. This is the foundation. Without it, no other layer has a stable referent. Section 3 sets out the system of record schema; section 4 covers profile, skills and blessings.

Layer two is Policy. It comprises the capabilities, scopes, lawful bases, consent receipts and rulebooks that declare, in code, not in committee minutes, what an agent is permitted to attempt and under what conditions. Section 5 covers fine-grained permissioning; section 11.3 introduces rulebooks as the codified, versioned, attestable expression of policy.

Layer three is Enforcement. It is the gateway plane in the path of every agent call, inbound and outbound. It is the only layer that is non-bypassable: every other layer is enforced through it. Without enforcement in the wire, identity, scope and policy are all advisory. Section 6 covers the policy enforcement plane in detail.

Layer four is Evidence. It is the tamper-evident audit log, the provenance graph, the deterministic replay and the chain of custody. Evidence makes enforcement provable; without evidence,

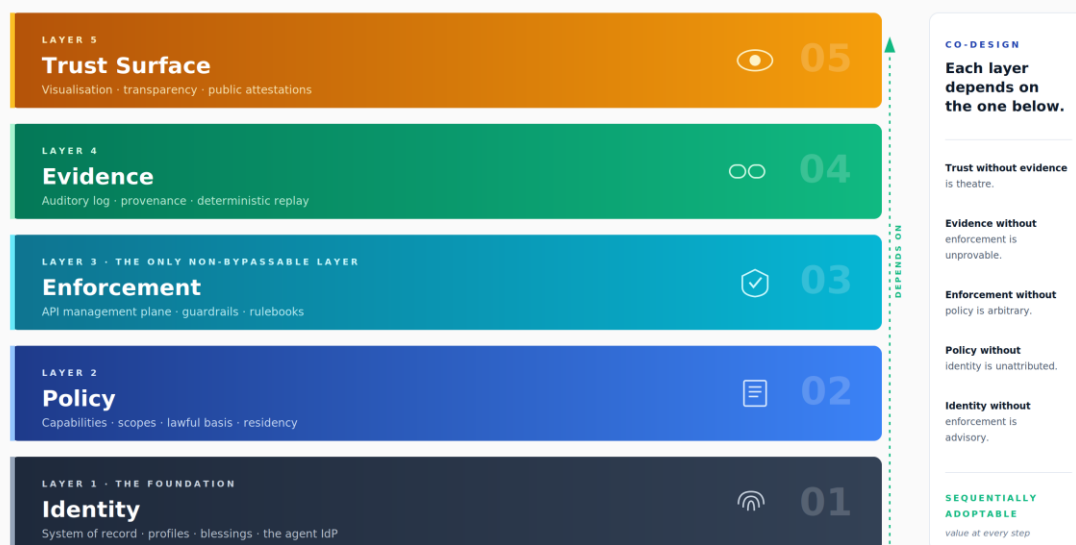
enforcement is asserted but not demonstrable. Section 7 sets out the audit layer, including the storage tiering required to balance retention with admissibility.

Layer five is Trust Surface. It is the visualisation, transparency and attestation surface presented to operators, auditors, customers and regulators. Without a trust surface, evidence is forensic-only and the framework's value is realised only after incidents. With a trust surface, evidence becomes operational, regulatory and commercial, a sales surface, a procurement differentiator, a reduction in insurance premium. Section 9 covers visualisation and operator surfaces; section 10.8 covers compliance as a product surface.

Identity without enforcement is advisory. Enforcement without evidence is unprovable. Evidence without a trust surface is forensic-only. Each layer earns its keep only when the others are in place. The framework's recommended adoption order, set out in section 11.10, reflects this dependency structure rather than any preference for a particular vendor stack.

FIGURE 1 · THE FIVE-LAYER MODEL

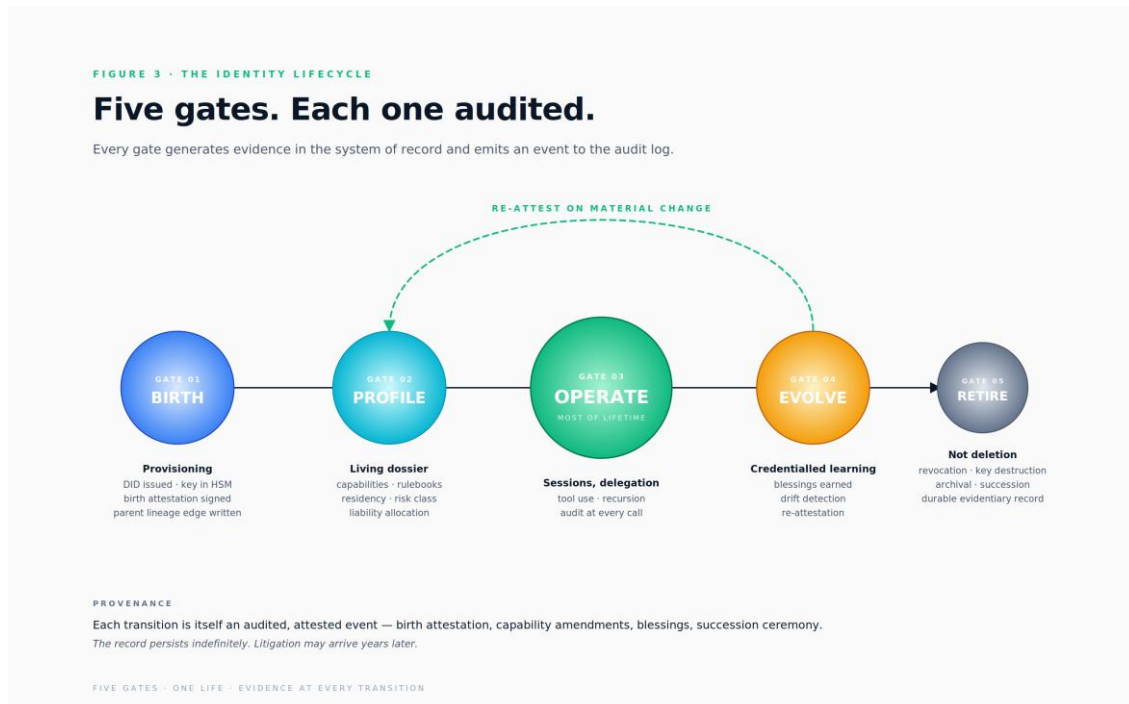
**Each layer is independently valuable.
Together they form the architecture.**



THE FRAMEWORK · five layers · sequentially adoptable · necessary but insufficient on its own

2. Identity Lifecycle

An agent's life passes through five gates: birth, profile, operate, evolve and retire. Each gate generates evidence in the system of record and emits an event to the audit log. The lifecycle is not a metaphor; it is the operational structure that makes every later property, federation, replay, ethics review, drift detection, possible. An agent that cannot be located in this lifecycle is, by definition, ungoverned.



2.1 Birth, provisioning

Every agent is provisioned, never spontaneous. Spontaneous agents, instances that come into existence without an authorising principal, are the primary failure mode of permissive orchestration frameworks, and the framework rejects the pattern at the foundation. Provisioning produces, at minimum, four artefacts. The first is a globally unique agent identifier, expressed as a Decentralized Identifier so that the identity is portable across organisations without requiring trust in any single registry. The second is a cryptographic key pair, with the private key stored in a hardware security module or cloud key management service scoped to the agent. The third is a birth attestation signed by the issuing authority, the organisation's agent identity provider, naming the human or service principal that authorised creation, the model and version, the declared purpose, and the rulebooks bound at birth. The fourth, where the agent has been spawned by another agent, is a parent lineage edge that records the provenance of the spawning event.

Provisioning is a write-once event. An agent's birth attestation is immutable; profile changes thereafter are amendments, not rewrites. This discipline is what makes the audit log evidentiary rather than narrative. An auditor reading the record three years later must be able to distinguish between

the agent as originally constituted and the agent as it became, and the framework's append-only treatment of the birth attestation makes that distinction trivially answerable.

2.2 Profile

Profile is the agent's living dossier. Where the birth attestation is fixed, profile evolves through credentialled events. Section 4 expands this in full, but at the lifecycle stage profile captures the agent's declared purpose and bounded scope, its initial capability set (which tools and which APIs it has been authorised to invoke), the rulebooks bound to it (regulatory, contractual and ethical), the residency to which the agent is pinned (which jurisdictions it operates within), the risk classification under EU AI Act framing (limited, high or prohibited), and the liability allocation between operator, provider, integrator and customer. Profile is the answer to "what can this agent do, and on whose authority", a question that recurs in every subsequent layer.

2.3 Operate

Operating is where most of the lifecycle's clock time is spent. In each session, the agent presents a short-lived, scope-bound credential to the policy enforcement plane. Each call is policy-checked, scope-bound and audited as it crosses the gateway. Recursive sub-tasks spawn child sessions whose lineage headers preserve the chain of authority back to the originating principal. Mode, human-led, human-intervened or autonomous, is recorded per action. Cost, latency and policy verdicts feed the visualisation surface in real time, allowing operators to see drift, anomaly or runaway as they begin rather than after they conclude.

2.4 Evolve

Agents learn. The framework's central discipline is that learning is treated as a credentialled event rather than a silent drift. Every blessing earned, every demonstrable instance of a capability successfully exercised, every benchmark passed, every reviewer approval granted, is signed by an attesting authority and recorded in the system of record. Material changes (a model upgrade, a prompt revision, the binding of a new tool) trigger re-attestation: the agent passes back through ethics review at a level proportional to its risk class. Drift detection runs continuously and is not a quarterly project. Capability misuse, output anomaly, cost spike and refusal-rate change are all monitored. Drift exceeding a threshold triggers a forced re-attestation gate; the agent does not continue to operate at full scope until the gate is cleared.

2.5 Retire

Retirement is not deletion. An agent in retirement has had all credentials revoked at the policy enforcement plane and across federated trust roots. Its private key has been destroyed, with the destruction itself attested by the hardware security module so that revival is cryptographically impossible. Its trajectory data is archived per the retention policy of every rulebook to which it was

bound, because litigation may arrive years after the agent has ceased operation and the framework's defence rests on the audit trail surviving.

Retirement may include a succession ceremony, a signed handover to a successor agent of skill bindings, retained memories (with the consent of any data subjects whose data underlies those memories), open delegations and customer relationships. Succession events are themselves audited; the successor agent's birth attestation references the predecessor's retirement attestation, and the lineage graph makes the continuity discoverable. Retirement is durable: the system of record persists indefinitely (or per legal retention), not as a live identity but as evidentiary history. An agent that has been retired and whose record has subsequently been deleted is an agent that cannot be defended in litigation; the framework therefore declines to support such deletion outside of structured legal-hold release.

3. Master System of Record , the Agent Identity Provider

The Agent Identity Provider is the master record. It is the canonical answer to the question "who is this agent, and on whose authority". Everything else in the framework hangs off it: capabilities are defined relative to it, audit events reference it, ethics reviews attest to it, federation requires its DIDs to be resolvable. An organisation that does not have a master record for its agents has no foundation on which to build the rest of the framework, no matter how sophisticated its other tooling.

3.1 The core schema

Each agent's record carries, at minimum, the following fields. A globally unique agent identifier, expressed as a Decentralized Identifier in a namespace that the issuing organisation controls. The issuing authority, the organisation, expressed as its own DID. The creation timestamp and the principal that authorised creation, with a reference to the change-management ticket or workflow that records the authorisation. The agent's status, drawn from a controlled vocabulary of provisioned, active, suspended and retired. The model in use, its provider, family, version and a fingerprint by which a specific deployment can be uniquely identified. The system prompt's hash and version chain, so that prompt drift is detectable. The agent's declared purpose, expressed in human-readable form because it must be reviewable by humans. The risk classification under EU AI Act framing. The residency to which the agent is pinned. The parent agent and child agents, where the agent has been spawned by or has spawned others. The set of granted capabilities. The set of bound rulebooks. The liability allocation between operator, provider, integrator and customer. A reference to the agent's bill of materials. References to the signing and encryption keys held in the hardware security module. The chain of attestations, birth, ethics review, blessings, each signed by its issuing authority and time-stamped.

This schema is not exhaustive; it is the floor. Specific deployments will extend it with sector-specific fields (clinical credentialing, financial-services attestations, public-sector security clearances). The framework's contribution is the requirement that the schema exists, that it is canonical, and that every other system in the agent ecosystem treats it as the source of truth rather than maintaining parallel records that drift.

3.2 The Agent Bill of Materials

Every agent has an immutable, signed bill of materials. The construct is the agent equivalent of the software bill of materials that has become standard in regulated software supply chains, and it serves the same purpose: producing a single artefact that names every component of which the system is composed, so that downstream parties, auditors, regulators, customers, can verify integrity and trace provenance.

The Agent Bill of Materials names the model provider, family, version and fingerprint; the system prompt's hash and the prompt's version chain; the tool list with versions and integration provenance; the skill bindings with their versions; the training-data summaries for any fine-tuned components; the third-party dependencies; and the bound rulebooks. It is signed by the issuing authority and stored alongside the agent's record, indexed by content hash so that any tampering is detectable.

The bill of materials is the artefact that regulators, auditors and downstream customers will request. It is also the input to deterministic replay, the property described in section 7.3 by which a past session can be reconstructed faithfully enough to answer the question "what would a reasonable agent have done under those conditions". Without the bill of materials, replay is fiction, and the litigation defence the framework promises collapses. Replay without the bill of materials is reconstruction from imperfect memory, which is not a defence at all.

3.3 Lineage as a first-class graph

Agents spawn agents. The identity provider therefore maintains the lineage graph as a primary data structure, not as a side-effect of audit log analysis. Every node in the graph knows its parent (who spawned this agent, on what authority), its children (which agents this one has spawned), its mentors (which agents have trained or attested to it), and its delegators (which humans or agents currently delegate to it). The graph is queryable in both directions and at depth: "show me every agent descended from agent X in the last twenty-four hours" is a single graph traversal, and it is exactly the question that gets asked in incident response, in legal discovery and in regulator inbound.

The framework's insistence on lineage as a first-class structure reflects a hard-won lesson from distributed systems: that information which is implicit in event logs is information which is unavailable in practice. By the time an investigation is under way, the latency of reconstructing lineage from raw events is incompatible with the operational tempo of the response. The graph must exist before it is needed.

3.4 Federation

The agent identity provider must federate. An agent born in one organisation must be presentable to customer agents, to third-party APIs, to regulators and to other agent ecosystems. Federation is not optional: an agent that cannot present its identity outside its home organisation is not a citizen of the agent economy but a cul-de-sac, and the value of the framework's other properties, capability, blessing, ethics review, is forgone at the boundary.

Federation rests on four established standards. Decentralized Identifiers (W3C DID Core 1.0) provide portable identifiers that any party can resolve to a public key without requiring a contract with the originating organisation. Verifiable Credentials (W3C VC 2.0) provide portable, signed, selectively-disclosed attestations of capability, blessing and ethics review, a customer can verify "this agent has been ethics-reviewed by Verax under Rulebook X" without needing to trust Verax's web service

uptime. SPIFFE and SPIRE provide workload identity at the infrastructure layer, complementing DIDs at the application layer. Cross-organisation trust roots are governed by mutual recognition agreements, the agent peering model, which make explicit which credential issuers each organisation accepts and under what conditions. Section 12 sets out the federation model in detail.

4. Profile, Skills and Blessings

Identity tells you who an agent is. Profile tells you what the agent can do, what it knows, and what it has earned the right to attempt. The framework distinguishes three components, capabilities, skills and blessings, because each is governed differently and each fails differently.

4.1 The three components

Capabilities are declared up front. They are the set of things the agent is permitted to attempt, expressed as narrow, composable permissions. Capabilities are enforced at the policy enforcement plane: an agent cannot exceed its declared capabilities at runtime, regardless of what its skills or its model might suggest. Skills are modular, versioned, attested units of operational know-how, workflows, domain models, interaction patterns, loaded per session and composable across agents. Skills declare the capabilities they require; an agent must hold those capabilities for the skill to be loaded. Blessings are earned over time. They are evidence of demonstrated competence, passed benchmarks, completed runs without incident, human approval on a class of action, ethics review for a specific deployment. Blessings are expirable and revocable; a blessing earned by an agent under one configuration is forfeit when the configuration materially changes.

4.2 Skills as versioned, signed artefacts

A skill is a unit of operational know-how. Verax's working studio operates with a catalogue of approximately forty-four skills today, ranging from procurement-document drafting through to operational handover sequences for smart-city deployments. In the framework, each skill is a signed artefact with a content hash, a semantic version, and a dependency graph that names every other skill or capability it requires. Skills are content-addressable, meaning that the assertion "agent X bound skill agent-sales-proposal at version 1.4.2" is a verifiable claim, not a vibe. Skill updates that introduce new tools, new capabilities or new external calls require re-attestation, because the change is material to the agent's risk profile. Updates that adjust prose, refactor logic without expanding scope, or fix bugs do not require re-attestation but are still version-tracked so that drift can be reconstructed.

4.3 Blessings, earned credentials

A blessing is what an agent has earned. It is issued by an attesting authority, a reviewer agent, a human approver, an automated benchmark suite, or an ethics committee, and it is signed as a Verifiable Credential so that it survives federation. Blessings are scoped: a blessing for "draft contracts in the United Kingdom" does not transfer to "draft contracts in Germany" without further attestation, because the underlying competence is not the same. Blessings are expirable: most expire on a fixed schedule or upon a material change to the agent (model upgrade, prompt revision). Blessings are revocable: a blessing can be withdrawn after an incident, after detected drift, or after a policy change that invalidates the original attestation. The audit log records every blessing earned, every blessing

exercised, and every blessing revoked. The system of record is the directory; the audit log is the journal.

4.4 Memory hygiene

Agents accumulate context. They build vector stores, summaries, learned facts, retrieved snippets. This accumulated context is data, governed by the data compliance section that follows, but it has dynamics specific to learning systems that warrant explicit treatment in the profile section.

Contamination is the first risk. A single bad embedding can poison retrieval for every subsequent call, because retrieval surfaces the closest match by similarity rather than by trust. The framework therefore requires that memory ingestion is checked at the boundary and that memory writes are reversible. Drift is the second risk. Memories diverge from primary sources over time as the underlying facts change but the embeddings do not. Memory entries must therefore carry source-of-truth references and re-validation schedules; an embedding more than ninety days from its source is, by default, treated as suspect.

Per-tenant isolation is the third risk. In multi-tenant deployments, memories must never cross tenants, a memory generated in the course of serving Customer A must never surface in a session serving Customer B. The framework requires this isolation to be enforced at the vector store, not at the agent, because application-layer isolation has the shelf-life of the developer who wrote it. Forgetting is the fourth risk: erasure of memory is operationally harder than erasure of primary data, because embeddings, summaries and derived insights must all be reachable. Crypto-shredding, the technique covered in section 10.4, is the operational answer. Catastrophic forgetting versus intentional forgetting is the fifth risk: model fine-tuning can inadvertently erase prior capabilities, and the framework treats this as a material change requiring re-attestation rather than as an internal model concern.

5. Fine-Grained Permissioning

Roles do not work for agents. The role abstraction was developed for organisations of humans whose jobs change rarely; it assumes a stable set of personas to which permissions can be attached and which then mediate access to systems. Agents have generative scope: their permissions need to compose dynamically, expand on legitimate need, and contract on completion. The framework therefore uses capability-based authorisation throughout, and treats the role construct as an antipattern when applied to agents.

5.1 Capabilities, not roles

A capability is a single, narrow permission. The naming convention is hierarchical and dotted, expressing the resource, the action, and the scope. The Verax reference uses identifiers such as `cap.attio.people.read`, `cap.attio.people.write`, `cap.ms365.mail.send.internal` and `cap.ms365.mail.send.external.gated`, the last of which expresses both the action and the requirement for a human gate before the action commits. Capabilities compose: a skill declares the capabilities it requires and an agent's profile declares which capabilities it has been granted. The policy enforcement plane enforces the intersection: a skill can only execute the capabilities granted to the agent that has bound it.

5.2 Scope-bound tokens

Authorisation is carried by short-lived, scope-bound credentials, JSON Web Tokens with proof-of-possession binding, or mutual TLS-bound credentials. Tokens carry the calling agent's identifier, the principal chain (the human or agent on whose behalf the call is being made), the subset of capabilities granted for this specific token (always less than or equal to the agent's profile capabilities), the declared purpose of the call (forming the lawful basis for any data processing), the consent receipt identifier where customer data is involved, the tenant scoping for multi-tenant operation, the operating mode, a short time-to-live measured in minutes, and a parent run identifier so that recursive sub-calls preserve trace continuity.

Tokens are minted by the identity provider, never issued long-lived, and never carried in environment variables, file system credentials or other persistent storage that an agent might inadvertently log. The framework treats credential persistence outside the gateway-mediated path as an antipattern; brokered credentials, described later in section 6, are the recommended pattern for third-party API access.

5.3 Just-in-time elevation

When an agent requires a capability outside its baseline grant, it does not silently acquire it. It emits an elevation request naming the capability, the reason, and the scope. The policy engine evaluates

the request: low-risk requests within rulebook are auto-approved, medium-risk requests are escalated to a human, and prohibited requests are denied at the gateway. Approved elevations mint a fresh, narrow, expiring token rather than amending the agent's baseline profile, and the elevation, the reason and the human approval (where any) are logged. Just-in-time elevation defeats capability creep, which is the most common failure mode in long-lived agents and the failure mode least likely to be detected by point-in-time audit.

5.4 Consent receipts

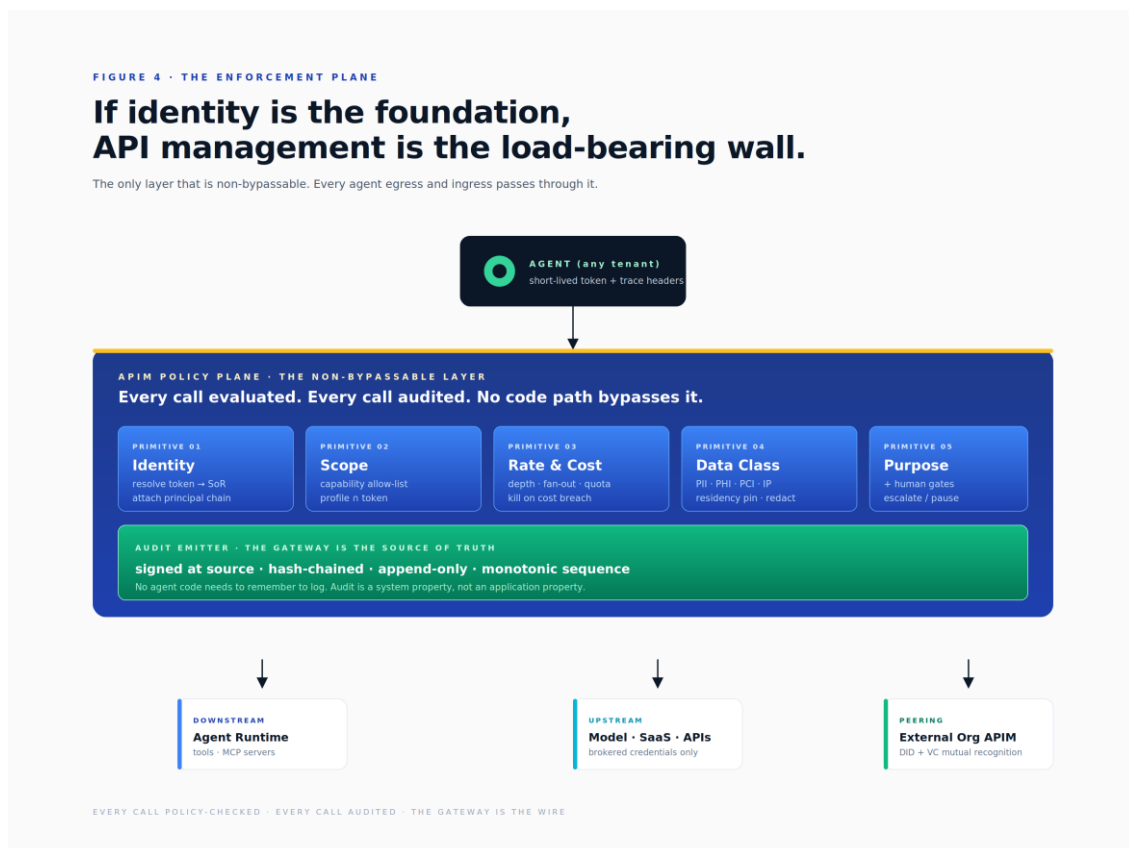
Where customer data is involved, every session is bound to a consent receipt: a signed, machine-readable artefact that names the data subject, the lawful basis under which their data is being processed, the purposes consented to, the retention period, and the right of withdrawal. Receipts are issued by the customer's identity domain, presented at session start, and referenced in every subsequent audit event. Without a valid receipt in scope, the policy enforcement plane rejects the call. This treatment of consent as a first-class artefact rather than a checkbox in a privacy policy is what allows the framework to satisfy GDPR Article 5 purpose-limitation obligations at the level of individual API calls rather than at the level of broad system documentation.

5.5 Human-in-loop gates

Capabilities can be configured to require a human gate even when the agent holds the relevant scope. Gates are defined by a combination of conditions: a capability pattern (any external-communication capability, for example), a data class (any call carrying personal information or payment card data), a risk score (an anomaly threshold drawn from the visualisation surface), a cost threshold (any single call exceeding a stated dollar amount, or any session whose cumulative cost exceeds a different threshold), and a rulebook clause (a specific regulatory rulebook may mandate a human gate on all autonomous external communications). Gated calls pause at the gateway, surface in the operator console, and proceed only after a signed approval from an authorised human reviewer. The approval, the approver and the timing are all logged. Documented refusals, calls that the agent itself declines, or that the human reviewer declines, are also logged, because the refusal and its reason are often more defensible in litigation than the action itself.

6. The Policy Enforcement Plane

If the identity provider is the foundation, the policy enforcement plane is the load-bearing wall. Without a gateway in the path of every call, inbound, outbound, peer-to-peer, identity, scope and audit are advisory. With a gateway, they become non-bypassable. The framework's most important architectural decision is therefore not which identity provider to use but where the gateway sits and what it enforces. The gateway plane is referred to throughout this paper as the policy enforcement plane, or simply APIM (API Management), reflecting the commercial category in which it is most easily procured; readers should understand the term to mean any equivalent gateway, mesh or sidecar that holds the same architectural position.



6.1 What the enforcement plane enforces

The enforcement plane is a composition of nine policy primitives. The first is identity binding: the gateway resolves the presented token to a system-of-record entry and attaches the agent identifier, principal chain, capability set, consent receipt identifier and tenant identifier to every downstream call as canonical headers. The second is the capability allow-list: a call is permitted only if the requested capability matches the agent's profile and the current token's scope. The third is quota and rate per identity: recursion guards, fan-out caps, calls per minute and maximum recursive depth limits. These are not ergonomic conveniences; they are the framework's defence against runaway agents at the wire. The fourth is cost capping per session and per agent. Cost is a behavioural risk indicator equal in

significance to security risk: a session whose cost trajectory diverges from the established baseline is, by definition, doing something it has not done before. The framework requires session-level kill switches that trigger on cost threshold breach, the same as on security policy breach.

The fifth primitive is geographical and data-residency pinning. The agent's residency must match the endpoint's residency; cross-border calls require an authorised transfer mechanism (Standard Contractual Clauses, an International Data Transfer Agreement, an adequacy decision, or sector-equivalence regimes such as those operated within the DIFC or ADGM free zones, where applicable). The sixth is schema and personally-identifiable-information inspection: payload validation, secret detection, redaction of personal data into hash-referenced encrypted storage, and classification tagging. The seventh is purpose binding: the token's declared purpose must match the endpoint's declared purposes, an enforcement that maps directly to GDPR Article 5(1)(b) purpose limitation. The eighth is the human-gate trigger described in section 5.5. The ninth is brokered credentials to third parties: the agent never sees the raw API key for an external service. The gateway holds the credential and signs requests on behalf of the agent identity, which means that an agent compromise does not leak vendor credentials and that vendor-credential rotation does not require touching every agent.

6.2 Trace propagation for recursive and multi-agent calls

Every call carries a structured set of headers that allow the call tree to be reconstructed. The W3C Trace Context standard provides the baseline traceparent header. Beyond the standard, the framework requires an agent-lineage header listing the ordered set of agent identifiers in the current call tree, a parent run identifier pointing to the orchestrating run, a principal chain naming the humans or agents whose authority underlies the call, and a tenant identifier for multi-tenant scoping. Headers are signed or HMAC'd at issue so that a downstream agent cannot fabricate a lineage by mutating in-flight headers; the gateway verifies signatures on receipt. Depth and fan-out bounds are enforced across the whole call tree, not per hop, because per-hop limits do not constrain the total cost or risk of a recursive operation.

6.3 Audit emission as a first-class output

Every call through the enforcement plane emits a canonical audit event capturing the request hash, the response hash, the latency, the cost, the policy verdict, the identity context, the headers and the classification tags. The event is signed at source and appended to the hash-chained audit log described in section 7. Crucially, no agent code needs to remember to log: the gateway is the source of truth, and the audit emission is a property of crossing the gateway rather than a property of agent virtue. This single architectural commitment, that audit is gateway-emitted rather than agent-emitted, is what makes audit reliable in the presence of buggy or malicious agents.

6.4 FinOps as a control surface

Cost is not solely an accountant's concern. In the framework, every agent carries a per-session budget envelope (with automatic kill on breach), a per-day envelope (with alarm thresholds before kill), a per-month operator envelope (used for chargeback and budget allocation), and cost-per-outcome attribution. Cost-per-outcome is a more useful metric than cost-per-token because it ties spend to the business decision the agent has been retained to support: a closed deal, an enriched record, a generated proposal. Underwriters of AI liability cover increasingly require this evidence to bind the policy at favourable terms. Cost discipline and security discipline are the same discipline at the gateway.

6.5 Multi-tenant policy partition

Where the platform serves many customers, the enforcement plane partitions per tenant across four dimensions. The identity namespace is partitioned so that no agent identifiers collide across tenants. The policy partition is enforced so that one tenant's permissions never bleed into another's evaluation. The quota partition is enforced so that one tenant's runaway agent cannot starve another tenant's legitimate operations. The audit partition is enforced so that no tenant can query another tenant's events. The partition is enforced at the gateway, not in application code, because the gateway is the only layer that is non-bypassable. Application-layer multi-tenancy is, in the long run, fiction.

6.6 Reference stacks

The framework is gateway-agnostic at the level of specification. Several commercial and open-source stacks satisfy the requirements; the table below characterises the leading options at the time of writing. The Verax reference deployment uses Azure API Management with an Open Policy Agent sidecar for declarative rulebook enforcement, chosen for adjacency to Microsoft 365 and Microsoft Graph, EU residency support, and procurement maturity in the markets Verax serves.

Stack	Strengths	Weaknesses
Azure API Management with Functions	Strong policy engine; native Microsoft 365 and Graph adjacency; mature enterprise procurement	Latency on cold paths; cost at high query rates
Kong with Open Policy Agent	Open source; rich plugin ecosystem; OPA enables declarative policy	Operator burden; OPA learning curve for policy authors
AWS API Gateway with Lambda Authorizer and WAF	Tight AWS integration; mature in regulated workloads	Vendor lock-in; cross-cloud patterns harder to express
Envoy with OPA via the ext_authz interface	Mesh-native; finest-grained control available; cloud-portable	Highest operator complexity; strong platform team required

Stack	Strengths	Weaknesses
Tyk	Multi-cloud; open-source core; strong analytics	Smaller community than Kong; fewer third-party integrations

7. The Audit Layer

The audit layer is the single source of evidentiary truth about what agents did, on whose authority, with what data, under what policy. It is the layer at which the framework either holds up under regulator inspection, customer audit and litigation discovery, or collapses. It is also the layer at which the most expensive shortcuts are taken in practice: organisations adopt identity, deploy a gateway, and then either skip the audit layer entirely or implement it with insufficient rigour to be admissible. The framework therefore treats audit as a first-class layer with its own design discipline, its own storage tiering, and its own admissibility requirements.

7.1 What gets logged

Every event captures, at minimum, a globally unique event identifier; a monotonic sequence number scoped to the tenant; the hash of the previous event, forming the chain that gives the log its tamper-evidence; the agent identifier and the parent run identifier; the principal chain and the tenant; the operating mode at the time of the action; the capability invoked and the endpoint addressed; the data class of the payload; the residency of the call; the declared purpose and the consent receipt identifier where applicable; the request and response hashes; the policy verdict; any guardrails triggered; the latency and the cost; any self-learning delta produced by the call; the human-gate state, including whether a gate was required and whether it was approved; and the event signature applied at the gateway. This schema is canonical: every event in every tenant of every deployment shares the same field set, allowing cross-tenant analytics, cross-deployment comparison, and uniform regulator response.

7.2 Tamper evidence and admissibility

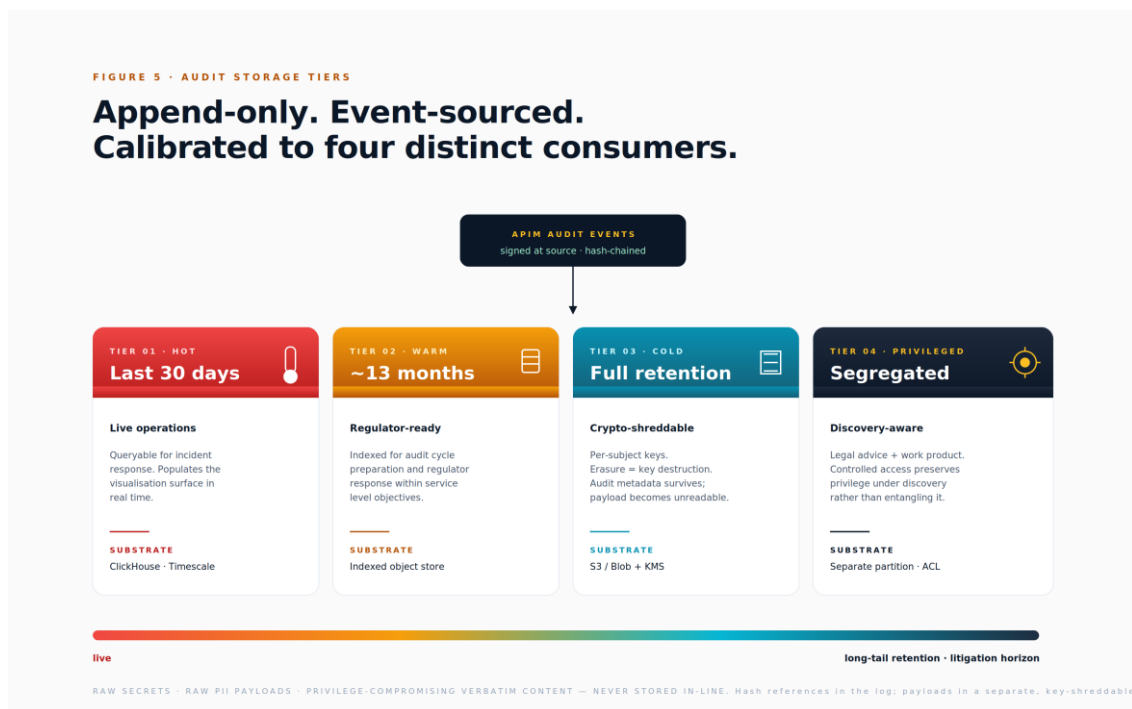
The audit log is append-only and hash-chained. Each event's record of the previous event's hash forms a Merkle-style transparency log structure: tampering with any event invalidates every subsequent hash, and the invalidation is detectable by any party in possession of the chain. Periodic checkpoints are signed by an external timestamp authority and, optionally, anchored to a public ledger or transparency log so that even the issuing organisation cannot retrospectively rewrite history without detection.

The combination of hash chaining, signed checkpoints, monotonic sequence numbers and gap detection at ingest constitutes the ingredients of admissibility under the rules of evidence in the major common-law and civil-law jurisdictions. In US federal practice, Federal Rules of Evidence 901 and 902 govern authentication and self-authentication of records; in English civil practice, Civil Procedure Rules Part 31 governs disclosure; equivalent provisions exist across the European Union. The framework's audit structure is designed to satisfy all three. Admissibility is not a property that emerges by accident from a well-engineered logging system; it is a property that is designed in from the outset, and the

cost of retrofitting it after a regulator inbound is substantially higher than the cost of building it correctly the first time.

7.3 Replay

The framework's most valuable property in litigation is deterministic replay. Given an audit trace plus the Agent Bill of Materials, any past session can be reconstructed: the original inputs from the logged request hashes (with payloads recoverable from encrypted storage), the original model, prompt and tool versions from the bill of materials, the original policy state from the rulebook version snapshot taken at session start, and the original learned state from the blessing snapshot at session start. Replay answers the question that the emerging legal doctrine of the reasonable agent will turn on: "would any reasonably configured agent have done the same thing under those conditions?" Without replay, this question is debated; with replay, it is answered.



7.4 Storage tiers

The audit log is too large and too long-lived to keep entirely on the hot path, and too important to store in a single tier. The framework specifies four tiers. The hot tier holds the last thirty days of events, queryable by live operations and feeding the visualisation surface. The warm tier holds thirteen months of events, indexed for regulator response, a regulator who arrives requesting twelve months of activity should be served from the warm tier directly. The cold tier holds the full retention period, encrypted with per-subject keys to support crypto-shredding (described in section 10.4) and stored in object storage at low marginal cost. The privileged tier is a separate partition for events containing legal advice or attorney work product, with controlled access maintained throughout the discovery

process so that legitimate legal-professional privilege is not entangled with general audit material and inadvertently waived.

7.5 What never goes in the audit log

Three categories of content are excluded from the log itself, even though their presence would be operationally convenient. The first is raw secrets, API keys, passwords, tokens, which are redacted at the gateway before any event is emitted. The second is raw personally-identifiable information payloads, which are replaced in the log by hash references to encrypted storage; the full payload remains recoverable through a controlled access path, but the audit log itself does not constitute a personal-data lake. The third is anything that would compromise legal-professional privilege if produced verbatim in discovery. Logs are evidentiary; they are also discoverable. Both must be true simultaneously, and the exclusions above are the price of preserving both properties.

8. Self-Learning Provenance

Agents that learn need their learning audited. Otherwise self-learning becomes laundering: an agent quietly absorbs capabilities that the framework did not authorise, the system of record no longer reflects reality, and the audit logs no longer mean what they appear to mean. The risk is not theoretical. Long-lived agents in production environments routinely accumulate context, prompt revisions, fine-tuning increments and tool integrations that, taken together, transform the agent into a system materially different from the one originally attested. Without provenance discipline, the agent ends up operating outside its ethics-reviewed envelope without any single change having appeared significant in isolation.

8.1 What counts as learning

The framework defines learning broadly. An updated system prompt or material change to in-context guidance counts as learning. The binding of a new skill, the addition of a new tool, or the grant of a new capability counts as learning. A new blessing earned counts as learning. Any change to model weights through fine-tuning counts as learning. A material update to the vector store, a new long-term memory of a kind not previously present, counts as learning. An updated rulebook binding counts as learning. Each of these is a material change event, and each requires re-attestation proportional to the agent's risk class. The threshold for materiality is deliberately low; the cost of treating a change as material when it was not is administrative, while the cost of treating a material change as immaterial is the loss of the audit trail's integrity.

8.2 The provenance chain

For every learning event, the framework requires six artefacts to be captured. The source: what evidence justifies the change, a run trajectory, a benchmark result, a human approval, a regulator update? The attester: who signs off on the change, a reviewer agent, a human approver, the ethics committee, an automated benchmark? The scope: what does the change apply to, this agent only, all of its descendants, the entire tenant? The diff: what is the precise before-and-after, expressed in a form a future auditor can read? The validation: what regression and red-team evidence accompanies the change, so that the change is not just attested but tested? The effective date and expiry: when is the change live, when does it expire, and when does it require review even before expiry? The provenance chain is itself stored as a signed sub-graph in the system of record, not buried in commit messages or version-control history that can be rewritten.

8.3 Preventing capability drift

Drift is the silent enemy. The framework specifies four mitigations. The first is periodic capability audits: does the agent still use what it has been granted? Capabilities not exercised within a defined window are revoked, on the principle that an unexercised capability is a latent attack surface rather

than a useful contingency. The second is behavioural shadow testing: a subset of live traffic is replayed against canary versions of the agent, and outputs are compared. Divergence between the canary and the production version is a signal that requires investigation. The third is adversarial probing: automated red-team agents continuously test the production agents' guardrails for prompt injection, capability misuse and data leakage. The fourth is the drift dashboard surfaced in the visualisation layer, rendering capability heatmaps and refusal-rate trends so that operators can see drift in motion rather than only in retrospect.

8.4 Preventing compliance laundering via fine-tuning

Fine-tuning is the most dangerous form of learning because it can erase or reshape behaviour without producing any prompt-level change visible to the operator. The framework therefore treats any fine-tune as the birth of a new agent, with a parent lineage edge to the prior version, rather than as an in-place update. This treatment preserves three properties that an in-place update would destroy: the integrity of the pre-fine-tune audit history (because the original agent's record is not retroactively edited), the ability to roll back (because the predecessor agent's configuration is fully preserved), and the obligation to re-attest (because the fine-tuned agent is, formally, a new entity that has not yet earned blessings). The discipline is administratively heavy and therefore tempting to short-circuit; the framework's experience is that organisations which short-circuit it produce audit records they cannot defend.

9. Visualisation and Operator Surfaces

You cannot govern what you cannot see. Without a visualisation layer, the audit data is forensic-only, useful after incidents but invisible to the operators whose decisions create the conditions for incidents in the first place. The framework therefore treats visualisation as a first-class deliverable rather than a user-interface convenience layered on at the end of a project. Visualisation is the surface at which operators detect drift, customers see compliance, regulators receive responses to inbound enquiry, and auditors find what they need without recourse to engineering teams. It is, in commercial terms, the layer at which the framework converts from a cost centre into a differentiator.

9.1 The four canonical views

The framework specifies four views, each addressing a distinct operator question and each backed by the same underlying event log.

The Identity Graph answers the question "who exists, and how are they related?". It renders every agent in the system, its parent and child lineage, the model and skill versions in use, and the blessings each holds. The view is largely static, navigated like an organisational chart with depth controls; it is the view a regulator opens first when asking "who are your agents?".

The Interaction Graph answers the question "what is happening right now?". It renders the live network of calls between agents, humans and systems, with edges weighted by call volume, cost or risk score. Graphistry's GPU-accelerated graph rendering is the reference renderer at scales above one hundred thousand edges, with Neo4j Bloom for narrative drill-down and Cytoscape.js for embedded views. The interaction graph is where runaway recursion becomes visible as a spike in fan-out, and where capability exfiltration becomes visible as an unexpected edge to an external endpoint.

The Trajectory or Trace View answers the question "what did this agent do in this session?". It renders a single run as a call tree with timeline, recursion depth, tool-use markers, human-gate pauses, and self-learning deltas. It is built on OpenTelemetry waterfall conventions augmented with agent-aware annotations, and it is the view incident responders open first.

The Capability Heatmap answers the question "which agents are over-permissioned?". It renders a per-agent skill matrix crossed with usage frequency and policy verdicts. It is the view that surfaces capability drift, dormant blessings and over-permissioned identities, and it drives the periodic capability-revocation cycle described in section 8.3.

9.2 The data pipeline

The visualisation surface reads from materialised projections of the canonical event log; it never blocks the gateway and never operates as a primary store. Events flow from the policy enforcement plane onto an event bus (Kafka or EventHubs), and from there into four downstream systems in parallel: a graph store (Neo4j or Memgraph) for the identity and interaction graphs, a trace store (Tempo or

Jaeger) for trajectory views, an analytics store (ClickHouse or BigQuery) for heatmaps and aggregate analytics, and an object archive for the cold tier described in section 7.4. The design is event-sourced and append-only; every projection is reproducible from the canonical log, which means that visualisations can be re-shaped or extended without re-instrumenting the gateway.

9.3 Operator workflows

The visualisation surface enables a distinct set of operator workflows that are difficult or impossible without it. These include: showing every agent that touched a specified client's data within a stated window; explaining why a specific agent escalated to a human at a specific point in a session; identifying agents holding capabilities they have not exercised within a defined window, ahead of the periodic revocation cycle; finding rogue recursion as graph anomalies (depth above N, fan-out above M); replaying a session with full fidelity to the conditions at the time; and simulating the impact of revoking a specific blessing across the population of agents that hold it. Each of these workflows is a question that is asked weekly in a mature operations function, and each becomes tractable only when the visualisation layer is properly co-designed with the audit and identity layers below it.

9.4 Public-facing transparency surfaces

The same data, scoped and redacted, becomes a customer-facing trust product. A customer can be presented with the precise list of agents that touched their account in a stated window, the authority on which each touched it, the lawful basis under which the data was processed, and the consent receipts that govern the processing. The customer can be presented with their data-processing record, downloadable on demand. The customer can review the agents on the framework's maturity ladder and see which controls are operational. Audit becomes a sales surface; compliance becomes a procurement differentiator. The same regulator-ready evidence base, surfaced through different access controls and visualisation conventions, becomes the trust portal that closes the deal.

10. Data Compliance, Sovereignty and Regulatory Alignment

Compliance is not bolted on. It is the property that emerges when identity, the policy enforcement plane, audit and visualisation are correctly co-designed. Treat compliance as a separate workstream and it becomes a perpetual catch-up project against shifting regulatory requirements; design it into the layers above and it becomes an operational property that requires only periodic review. The framework treats every audit event as the answer to a single composite question: who, acting for whom, under what lawful basis, on what data class, in what jurisdiction.

10.1 Data classification at the gateway

Every payload is tagged at the policy enforcement plane with a classification drawn from a controlled vocabulary that runs from public through internal, confidential, personally-identifiable information, and finally to the most-sensitive categories such as protected health information, payment card data and intellectual-property secrets. Tags drive policy: protected health information never leaves its region of origin without explicit transfer authorisation, payment card data never reaches a generative model, intellectual-property secrets block third-party model egress entirely. Tags are captured in the audit event so that the visualisation layer can render data-class heatmaps and so that regulators can be answered in seconds rather than weeks: the question "did any protected health information leave the tenant in the last quarter?" becomes a query rather than a forensic project.

10.2 Lawful basis and purpose limitation

Every capability is bound to a declared lawful basis under the regime that applies to the processing, GDPR Articles 6 and 9, the UK Data Protection Act, the UAE Personal Data Protection Law, HIPAA's treatment-payment-operations framing, or the equivalent in other jurisdictions. Capability scope equals purpose: the same operation under a different lawful basis is, formally, a different capability. The policy enforcement plane rejects calls that fall outside the declared purpose of the issued token, the property the framework refers to as purpose-binding tokens. Consent receipts are signed, machine-readable artefacts referenced in every audit event that involves consent-based processing. The combination of purpose-bound tokens and consent receipts is what permits the framework to satisfy GDPR Article 5(1)(b) at the resolution of individual API calls.

10.3 Data residency and sovereignty

Agents are pinned to a residency at the level of the system of record (the agent.residency field). The policy enforcement plane blocks cross-border egress unless an authorised transfer mechanism is present in the call's metadata: Standard Contractual Clauses, an International Data Transfer Agreement, an adequacy decision, or sector-equivalence regimes such as those operated within the

DIFC or ADGM free zones, where applicable. Vector stores, audit logs, model endpoints and key material are all region-pinned. A sovereign deployment profile exists for government and regulated clients in domains such as smart-city infrastructure, defence-adjacent operations and sovereign healthcare. The sovereign profile comprises an air-gapped agent identity provider together with on-soil policy enforcement plane and audit, and it is offered explicitly to clients whose regulatory or political environment requires that no agent operating on their data, and no record of that operation, ever crosses a border.

10.4 The retention–erasure paradox and crypto-shredding

There is a structural tension between the right of erasure under GDPR Article 17 and the audit-log retention obligations under the EU AI Act, which require retention of at least six months for high-risk systems and substantially longer in sectoral overlays. Naive implementations pick one side of the tension and breach the other. The framework's answer is crypto-shredding: data is encrypted at rest with per-subject keys (one key per data subject, per tenant, per data class), and erasure is performed by destroying the relevant key rather than by deleting the underlying ciphertext. The audit metadata, who acted, when, under what verdict, survives, while the payload becomes unreadable forever. Per-data-class retention policies are enforced by the audit sink rather than by ad-hoc cleanup jobs, which would otherwise be a perennial source of audit-log integrity failures. Crypto-shredding is also the operational answer to memory-hygiene erasure described in section 4.4.

10.5 EU AI Act alignment

For most agent products operating in or serving the European Union, the EU AI Act is the binding regulator. The table below maps the Act's principal obligations to the framework sections that satisfy them.

AI Act obligation	Framework section satisfying it
Risk classification (limited, high, prohibited)	Section 3, system of record field risk_class
Technical documentation	Section 3 system of record together with the Agent Bill of Materials in section 3.2
Log retention of at least six months	Section 7 audit layer with the storage tiering described in section 7.4
Human oversight	Section 1.1 modes, section 5.5 gates, and section 11.4 ethics governance
Accuracy, robustness and cybersecurity metrics	Section 8 self-learning provenance and section 11.2 guardrail layers
Post-market monitoring	Section 9 visualisation and section 8 drift detection

AI Act obligation	Framework section satisfying it
General-purpose AI obligations (training data summary, copyright, evaluation)	Section 3.2 Agent Bill of Materials

10.6 Sector overlays

Sectors layer additional obligations onto the baseline. In healthcare, HIPAA, the NHS Data Security and Protection Toolkit, and equivalents such as the UAE Department of Health requirements add classification of protected health information at the gateway, restriction of vendor relationships to those bound by Business Associate Agreements (or local equivalents), and enforcement of the minimum-necessary rule as a capability scope rather than as a documentation exercise. In financial services, the Financial Conduct Authority's senior management arrangements, systems and controls handbook, the Monetary Authority of Singapore's regulatory expectations, ADGM and DIFC requirements, PCI-DSS, and SR 11-7 model risk management require operational-resilience treatment of agents and the recognition that the model itself is a fourth-party risk. In public sector and smart-city contexts, procurement-transparency rules, Algorithmic Impact Assessments, and public registers of automated decisions create additional disclosure obligations that the framework's visualisation surface is designed to satisfy. Cross-cutting frameworks, SOC 2, ISO/IEC 27001, ISO/IEC 27701, ISO/IEC 42001 and the NIST AI Risk Management Framework, map directly to controls in the framework, with the audit log providing the evidence base.

10.7 DPIA and Algorithmic Impact Assessment per agent class

The framework provides template Data Protection Impact Assessments and Algorithmic Impact Assessments per agent archetype, autonomous, human-intervened, human-led, that are regenerated automatically on material model or capability change. The assessments are stored against the agent's record in the system of record, where they live alongside the agent rather than in a separate compliance repository that drifts out of date. A DPIA in the framework is not a one-off project artefact but a living document with an explicit re-validation trigger; the framework's experience is that DPIAs treated as project artefacts have a half-life of approximately one year in a moving regulatory environment.

10.8 Compliance as a product surface

The customer-facing transparency portal extends to a downloadable data-processing record, the agent inventory, audit excerpts within the customer's authorised scope, retention status across the data-class spectrum, and erasure receipts for any erasure request the customer has made. Regulatory burden becomes a trust differentiator. The argument made to the executive sponsor is that compliance investment funded as a defensive cost yields ongoing operational returns when the same

investment is exposed as a customer-facing capability, and that the marginal cost of doing so is small once the underlying audit and visualisation layers are correctly built.

11. Risk, Ethics, Guardrails and Litigation Defence

The framework set out so far is necessary but not sufficient. Identity, enforcement, audit, visualisation and compliance produce the conditions for governable agent operation, but they do not, by themselves, constitute a posture on risk. Without a deliberate position on risk taxonomy, ethical governance, guardrail layering, continuous adversarial testing and litigation readiness, the identity and audit machinery is ornamental when subpoenas arrive. This section sets out the framework's posture on each.

11.1 Risk taxonomy

The framework operates a single risk register with six classes; parallel registers fragment the risk picture and create gaps at the boundaries.

Risk class	Representative examples
Behavioural	Hallucination; prompt injection; goal drift; sycophancy; tool misuse; recursive runaway
Identity	Stolen tokens; agent impersonation; capability creep; lineage forgery
Data	Leakage; contamination of training or memory; training-data poisoning; learned-secret retention
Operational	Cost runaway; cascading failures across delegated agents; third-party model outage
Legal and regulatory	Unlawful processing; automated-decision violations; sectoral breach; intellectual-property infringement in outputs
Reputational and ethical	Discriminatory outputs; manipulation; dark patterns

Each class is mapped to controls in sections 3, 6, 7, 9 and 10. The single-register discipline is what allows risk-acceptance decisions to be made coherently across categories that, in legacy frameworks, are owned by different functions and reconciled only at board level after material exposure.

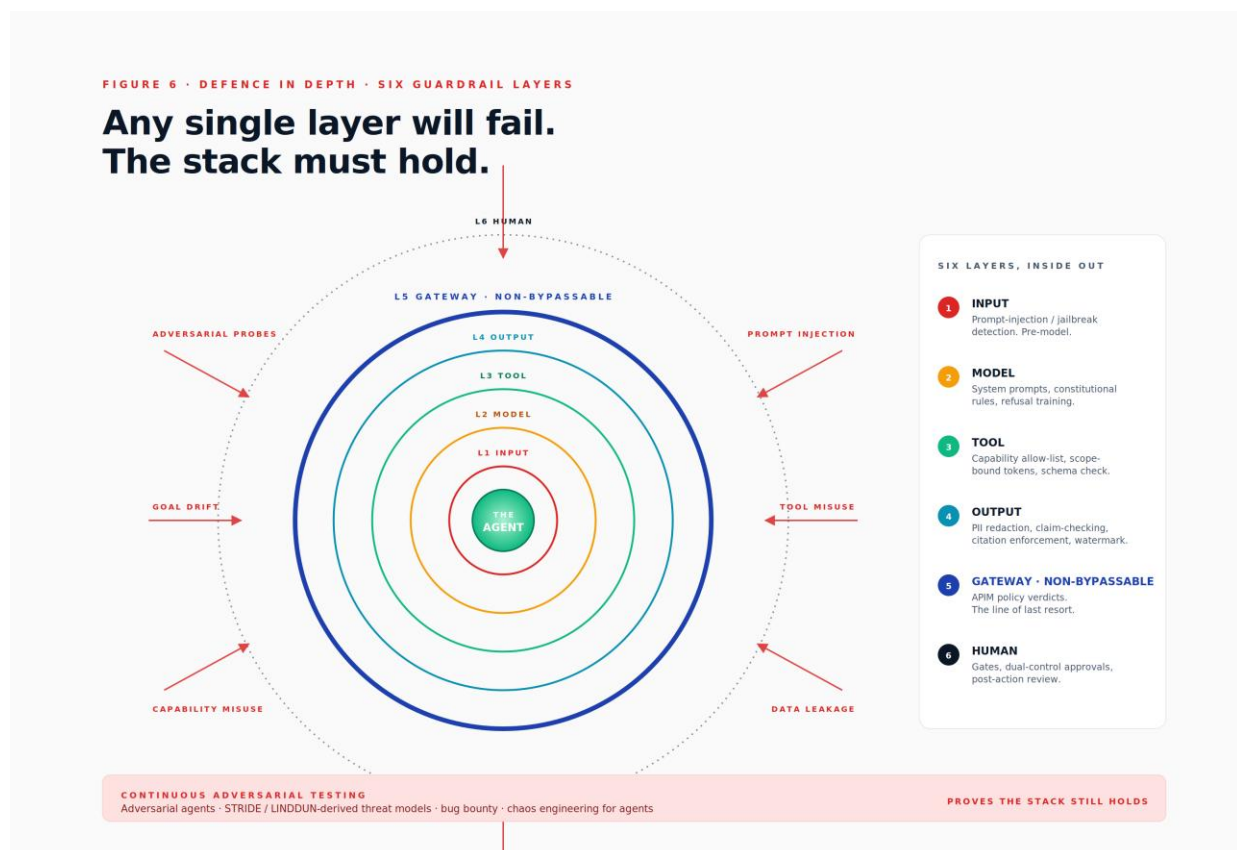
11.2 Defence in depth, six layers of guardrail

No single guardrail layer is reliable on its own. Each will fail, eventually, in one direction or another; the question is whether the stack as a whole still holds when any single layer has failed. The framework specifies six layers.

- **Input guardrails.** Prompt-injection and jailbreak detection at the entry of any user-controlled content into the agent's context.

- **Model guardrails.** System prompts, constitutional rules, refusal training built into the model itself or applied at inference time.
- **Tool guardrails.** Capability allow-lists, scope-bound tokens, schema validation at the tool boundary.
- **Output guardrails.** Personally-identifiable-information redaction, claim-checking, citation enforcement and watermarking applied to model outputs before they leave the agent.
- **Gateway guardrails.** Policy verdicts at the policy enforcement plane, the only non-bypassable layer in the stack.
- **Human guardrails.** Gates, dual-control approval requirements, and post-action review.

Continuous adversarial testing, described in section 11.6, is what proves the stack still holds in the face of evolving attack patterns. A guardrail stack that has not been adversarially tested in the last quarter is, by working assumption, weaker than its documentation suggests.



11.3 Rulebooks, codified, versioned, attestable

Vibes do not survive depositions. Ethical principles expressed as committee-meeting minutes, internal policy documents or values statements provide insufficient grip when an agent's behaviour is being interrogated three years after the fact. The framework therefore introduces the rulebook construct: a codified, versioned, attestable statement of operational rules with enforcement-layer binding and evidence-event binding.

Each rulebook names the source authority on which it rests (a HIPAA section, a GDPR Article, a contractual clause, an internal policy reference), the capabilities to which it applies, the enforcement layer at which it is enforced (the policy enforcement plane, the model layer, the tool layer or the human layer), and a set of assertions. Each assertion is a testable statement, "protected health information fields requested must match declared purpose codes", or "protected health information never crosses a tenant boundary", bound to the audit event that constitutes evidence of compliance. Rulebooks declare any human-gate requirements they impose, and they declare a review cycle (typically quarterly for live operational rulebooks). Rulebooks live in the system of record alongside agents; they are not committee minutes.

11.4 Ethics governance, three tiers

The framework specifies three tiers of ethical governance, mapped to the operational tempo at which each tier can usefully act. The Board AI Risk Committee meets quarterly, sets the organisation's risk appetite, approves high-risk deployments, and owns the relationship with the insurance underwriters. The AI Ethics Committee is cross-functional, engineering, legal, product, security, customer-facing, and meets at the cadence required to review high-risk agents pre-launch and to approve material rulebook updates. The Model Risk Officer is operational and continuous: this role owns the drift dashboards, escalates anomalies, and acts as the standing point of contact between operations and the committees above. The structure maps directly to ISO/IEC 42001, the NIST AI Risk Management Framework, and the EU AI Act's Article 17 governance expectations. Ethics in the framework is procedural rather than aspirational, a posture, not a poster.

11.5 Behavioural defence in litigation

Audit logs become discoverable evidence. They will help or hurt; they cannot be neutral. The framework's audit structure is designed to produce evidence that helps when the underlying conduct was sound and that fails gracefully when the underlying conduct was not.

Admissibility requirement	Framework satisfaction
Authenticity	Events signed at source; log hash-chained; checkpoints signed by external timestamp authority
Reliability	Automated capture (no human-edited gaps); trusted timestamps; documented schema with version control
Chain of custody	Logs about logs, every export, access and redaction is itself logged in the privileged audit partition
Completeness	Monotonic sequence numbers; gap detection at ingest; reconciliation alerts on detected gaps

Admissibility requirement	Framework satisfaction
Determinism	Agent Bill of Materials together with deterministic replay, allowing sessions to be reconstructed faithfully

The reasonable agent standard. An emerging legal doctrine, parallel to the long-established reasonable person doctrine in tort and the reasonable professional standards used in medical and financial litigation. Defence under the reasonable agent standard rests on showing four elements: a declared purpose and lawful basis; documented guardrails appropriate to the risk; human-oversight points actually exercised at the relevant moments; and a record showing that any deviation from policy was anomalous rather than habitual. The framework above produces exactly this evidence, but only if it has been operated, not merely deployed. Compliance theatre produces audit trails that fail under cross-examination; operating discipline produces audit trails that close inquiries before they escalate.

Documented refusal as a shield. When an agent declines an action, the refusal and its reason are logged. The refusal is often more defensible than the action: it proves that judgement was exercised, that the guardrail layer functioned as intended, and that the agent acted within its documented constraints. The framework treats refusals as evidence of correct operation rather than as anomalies to be investigated.

Privilege and data-room hygiene. Audit logs may be produced in discovery. They must be structured so that legitimate legal-professional privilege is segregated rather than entangled. Privileged events are written to the privileged partition described in section 7.4, with controlled access designed to preserve privilege under the rules of the relevant jurisdiction. Inadvertent waiver of privilege through commingled audit material is a foreseeable failure mode that the framework's storage tiering is specifically designed to prevent.

11.6 Continuous red-teaming

Pre-launch ethics review is necessary but not sufficient. The framework requires continuous adversarial testing across four modes. Adversarial agents probe live agents for prompt injection, capability misuse and data leakage, with their findings feeding into the rulebook update cycle. Threat models adapted from STRIDE and LINDDUN are extended for agentic contexts: spoofed identity (agent impersonation), tampered tool output, repudiation of recursive calls, information disclosure via memory contamination, denial of service via fan-out, elevation of privilege via capability creep. Bug bounty programmes for agents, third-party researchers paid to break them, scoped and rules-of-engagement bound, produce findings the internal team will not surface alone. Chaos engineering for agents, deliberate injection of tool outages, model outages, policy changes, validates that graceful degradation is a property of the system rather than an aspiration. The output of red-teaming feeds

the self-learning provenance section: every finding becomes either a rulebook update, a guardrail change or an explicit acceptance of residual risk.

11.7 Liability allocation

The four parties to an agent deployment, operator (the party deploying the agent), provider (the party providing the underlying model), integrator (the party assembling the agent), customer (the party whose data is processed), must have their responsibilities allocated explicitly and recorded in the agent's record. Contracts must mirror the EU AI Act's role definitions, and indemnity clauses should be keyed to which party owns which guardrail layer in the stack described in section 11.2. The framework captures the liability model per agent in the system of record so that the allocation is operationally retrievable rather than buried in the contracts repository.

11.8 Insurance and risk transfer

The emerging market in AI liability cover, Munich Re's offerings, certain Lloyd's syndicates, US-domiciled specialists such as Vouch and Armilla, and others, increasingly requires audit-log evidence to bind cover at favourable terms. Premium reductions are tied directly to the maturity of the policy enforcement plane, audit and risk controls described in sections 6, 7 and 11. The framework, in commercial terms, pays for itself: the difference between unbound exposure and bound exposure at favourable terms substantially exceeds the implementation cost of the audit and enforcement layers in nearly every realistic scenario.

11.9 Incident response and forensics

The framework specifies a five-stage incident response model. Detection is driven by anomaly signals from the visualisation surface, customer reports and regulator inbound. Containment is exercised via the kill-switch at the policy enforcement plane: revoke the agent's token, halt the parent run, freeze the descendant agents, block the tenant if necessary. Forensics rests on the combination of replay, the log timeline and the interaction-graph snapshot at the moment of the incident. Disclosure follows jurisdiction-specific playbooks: GDPR's seventy-two-hour breach notification window, the EU AI Act's serious-incident reporting requirements, customer-contractual obligations, and any voluntary public disclosure decisions. The post-mortem feeds back into the rulebook, the guardrail layer, the model card and (where appropriate) board notification, closing the loop that the self-learning provenance section is designed to keep auditable.

11.10 Practicalities and realities, the maturity ladder

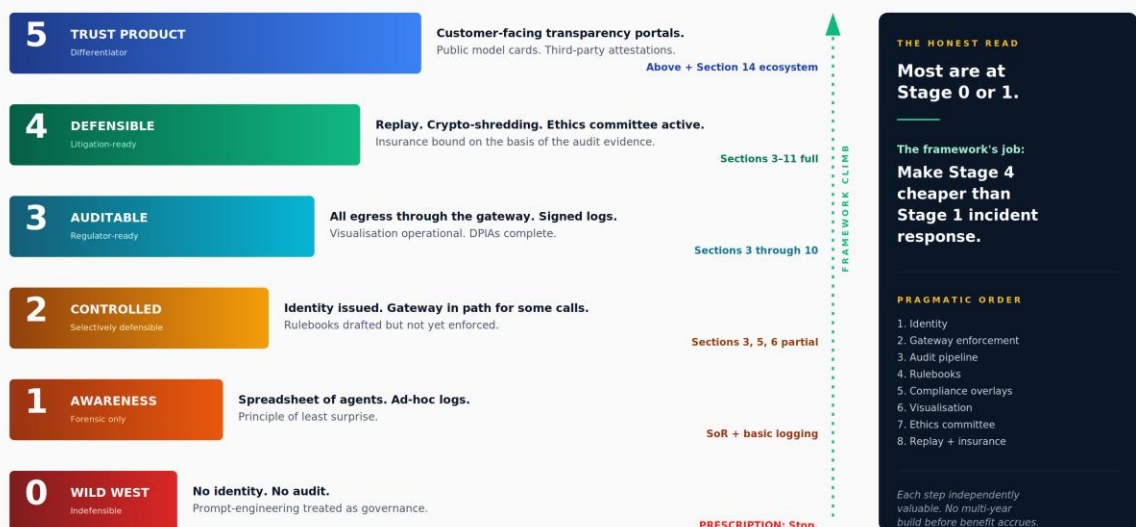
Most organisations, including most AI-native start-ups, are at the lower end of the maturity ladder set out below. The framework's commercial argument is that stage four is cheaper than stage one incident response, and that the path between them is incremental rather than monolithic.

Stage	Reality on the ground	What the framework demands
0 , Wild West	Agents in production; no identity, no audit; prompt-engineering as governance	Stop
1 , Awareness	Spreadsheet of agents; ad-hoc logs; principle of least surprise as an unwritten rule	Section 3 system of record plus basic logging
2 , Controlled	Identity issued; policy enforcement plane in path for some calls; rulebooks drafted	Sections 3, 5 and 6 partial
3 , Auditable	All egress through the policy enforcement plane; signed logs; visualisation operational; DPIAs done	Sections 3 to 10
4 , Defensible	Replay; crypto-shred; ethics committee; insurance bound; regulator-ready	Sections 3 to 11 in full
5 , Trust product	Customer-facing transparency portals; public model cards; third-party attestations	Sections 3 to 11 plus the ecosystem participation in section 14

FIGURE 7 · THE MATURITY LADDER

Six stages. One climb.

Most organisations — including most AI-native start-ups — are at Stage 0 or Stage 1.



Pragmatic adoption order. Identity (section 3) first. Policy enforcement plane (section 6) second. Audit (section 7) third. Rulebooks (section 11.3) fourth. Compliance (section 10) fifth. Visualisation (section 9) sixth. Ethics committee (section 11.4) seventh. Replay and insurance (sections 11.5 and 11.8) eighth. Each step is independently valuable; each unlocks the next.

Control-effectiveness testing. Controls that verify controls, red-team agents, audit-log audits, rulebook drift detection. Without this layer, the rulebooks rot, the ethics committees become rubber-stamps, and the audit trails accumulate without ever being interrogated. The discipline is unglamorous and expensive to neglect.

12. Multi-tenancy, Federation and Cross-Organisation Agent-to-Agent Operation

A framework that only works inside a single organisation cannot govern the agent economy. Real ecosystems are multi-tenant inside and federated outside, and increasingly the most valuable interactions are agent-to-agent across trust boundaries, a procurement agent at one organisation negotiating with a sales agent at another, a regulatory-reporting agent at a financial institution interacting with a supervisory agent at the regulator. Each of these crossings carries the full weight of identity, scope, audit, compliance and liability concerns that this paper has set out, and the framework's federation model is what allows those concerns to be addressed without re-litigating them at every boundary.

12.1 Multi-tenancy as a cross-cutting concern

Multi-tenancy is not a section of the framework but a property each layer must hold. The table below summarises the per-layer obligation.

Layer	Tenant isolation requirement
Identity (section 3)	Tenant-scoped namespaces; no cross-tenant identifier collision
Profile (section 4)	Memories, blessings and skills scoped per tenant
Permissioning (section 5)	Tokens carry the tenant identifier; capabilities never cross tenants
Policy enforcement (section 6)	Policy partition; quota partition; routing partition
Audit (section 7)	Per-tenant log streams; no cross-tenant query without explicit cross-tenant authority
Compliance (section 10)	Per-tenant residency; per-tenant rulebooks; per-tenant crypto-shred keys
Visualisation (section 9)	Operator scoping; customers see only their own data

Enforcement is at the gateway, not in application code. Application-layer isolation has the same shelf-life as the developer who wrote it; gateway-layer isolation has the shelf-life of the gateway itself, which is substantially longer and substantially more reviewable.

12.2 Federation between trust roots

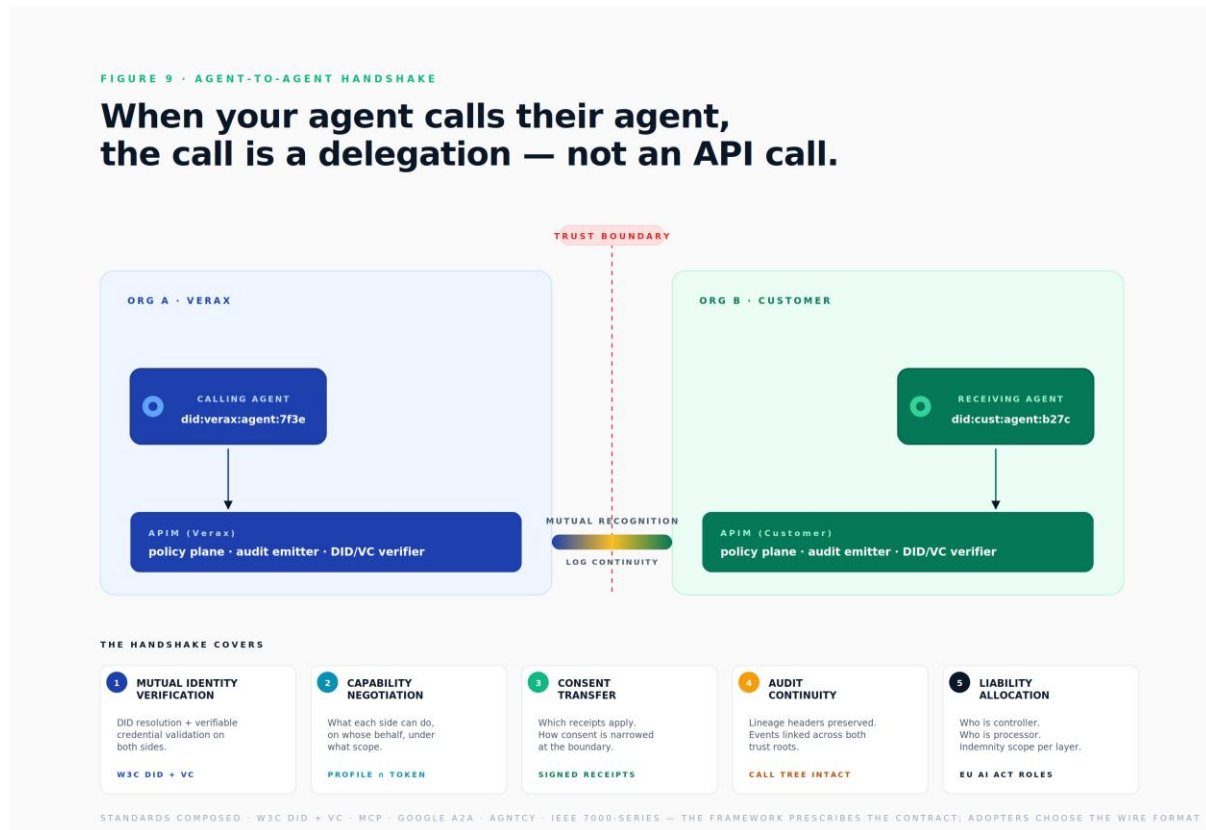
Cross-organisation operation rests on three primitives, each backed by a mature standard. Decentralized Identifiers, defined by the W3C DID Core 1.0 specification, provide portable identifiers that any party can resolve to a public key without contacting the originating organisation; the

framework is method-agnostic, with reference deployments using established methods such as did:web or did:key, and a custom method permitted where an organisation requires self-sovereign control of its registry. Verifiable Credentials, defined by the W3C VC 2.0 specification, provide portable, signed, selectively-disclosed attestations of capability, blessing and ethics review. A customer can verify the claim "this agent has been ethics-reviewed by Verax under Rulebook X" without trusting Verax's web service uptime or even Verax's continued existence. SPIFFE and SPIRE provide workload identity at the infrastructure layer, complementing DIDs at the application layer. Together, the three primitives allow an agent to present its identity and its earned credentials to any counterparty in the federation, and they allow any counterparty to verify those presentations without bilateral integration work.

12.3 Agent-to-Agent protocols

When an agent at one organisation calls an agent at another, the call is not a normal API call. It is a delegation, with all of the identity, scope, audit and compliance machinery riding on it. The handshake covers five distinct concerns. First, mutual identity verification: each side presents its DID and any required Verifiable Credentials, and each side resolves and verifies the other's. Second, capability negotiation: the calling agent's request is evaluated against the receiving agent's permitted operations on behalf of the caller, and a scope of action is agreed. Third, consent transfer: any consent receipts that bind the calling side's data must be presented and narrowed (not expanded) at the boundary. Fourth, audit continuity: lineage headers are preserved across the trust root, and audit events on the receiving side are linked to the calling side's events through the agent-lineage header so that incident response can reconstruct the full call tree. Fifth, liability allocation: the handshake records, in machine-readable form, which party is the controller of the data, which is the processor, and which indemnifies which under defined incident scenarios.

The wire-level protocols in this space are emerging rapidly. Anthropic's Model Context Protocol, Google's Agent-to-Agent specification, the AGNTCY initiative, the IEEE 7000-series and the W3C Verifiable Credentials working group are all relevant. The framework prescribes the contract, what must be exchanged, what must be verified, what must be logged, rather than the wire format. Adopters are free to choose the protocol that fits their stack, and the framework's audit and federation requirements are satisfiable across all of them.



12.4 Cross-organisation policy enforcement plane peering

Policy enforcement planes peer like Border Gateway Protocol routers. Each declares its accepted credential issuers, its accepted DID methods, and the rulebooks under which it operates. A peering agreement between two organisations is the operational manifestation of a contract, machine-readable, version-controlled, auditable, and the agreement records, at minimum: the identifiers the peering covers, the duration of the peering, the consent-transfer rules at the boundary, the audit-continuity expectations, and the liability allocations. Peering agreements are a form of compliance artefact in their own right; in mature deployments they are reviewed by both organisations' ethics committees before commitment.

12.5 Standards and conformance map

The framework references and aligns to a substantial set of existing standards rather than reinventing them. The table below summarises the principal mappings.

Concern	Relevant standards	Where in the framework
Identity	W3C DID, OAuth 2.1, OIDC, SPIFFE	Sections 3 and 5
Credentials	W3C VC, SD-JWT, OpenID4VC	Sections 4 and 12
Tracing	W3C Trace Context, OpenTelemetry	Sections 6 and 7

Concern	Relevant standards	Where in the framework
Audit	RFC 8392 CWT, RFC 9162 Transparency Logs	Section 7
AI management	ISO/IEC 42001, NIST AI RMF	Section 11
Data	ISO/IEC 27001, ISO/IEC 27701, GDPR, CCPA	Section 10
Sectoral AI	EU AI Act, US OSTP AI Bill of Rights	Section 10
Tool use	MCP, A2A protocols, OpenAPI 3.1	Sections 5 and 12

A formal conformance matrix lives in the reference implementation. The summary above is the high-level orientation; readers seeking implementation guidance should consult the conformance suite once it is published.

13. Reference Architecture, Developer Experience and Golden Paths

A framework that is harder to follow than to ignore will be ignored. The reference architecture and the authoring developer experience are not afterthoughts; they are the difference between a paper that ships and a paper that decorates a website. The framework's most effective enforcement is at the build-time CI gate, not at runtime, and the build-time gate is only effective if the path of least resistance for developers is the compliant path. The sections below describe the Verax reference deployment of the framework, the developer-experience commands that scaffold a compliant agent in seconds, and the staged adoption sequence by which a small team can reach a defensible baseline within roughly one quarter.

13.1 Reference architecture

The Verax reference deployment composes the framework's layers as follows. The operator and customer-facing interface, the Verax Command Center, is built on Next.js with a Supabase backend and surfaces the four canonical views described in section 9: Identity Graph, live Interaction Graph, trajectory and trace view, and Capability Heatmap, together with the customer-facing trust portal.

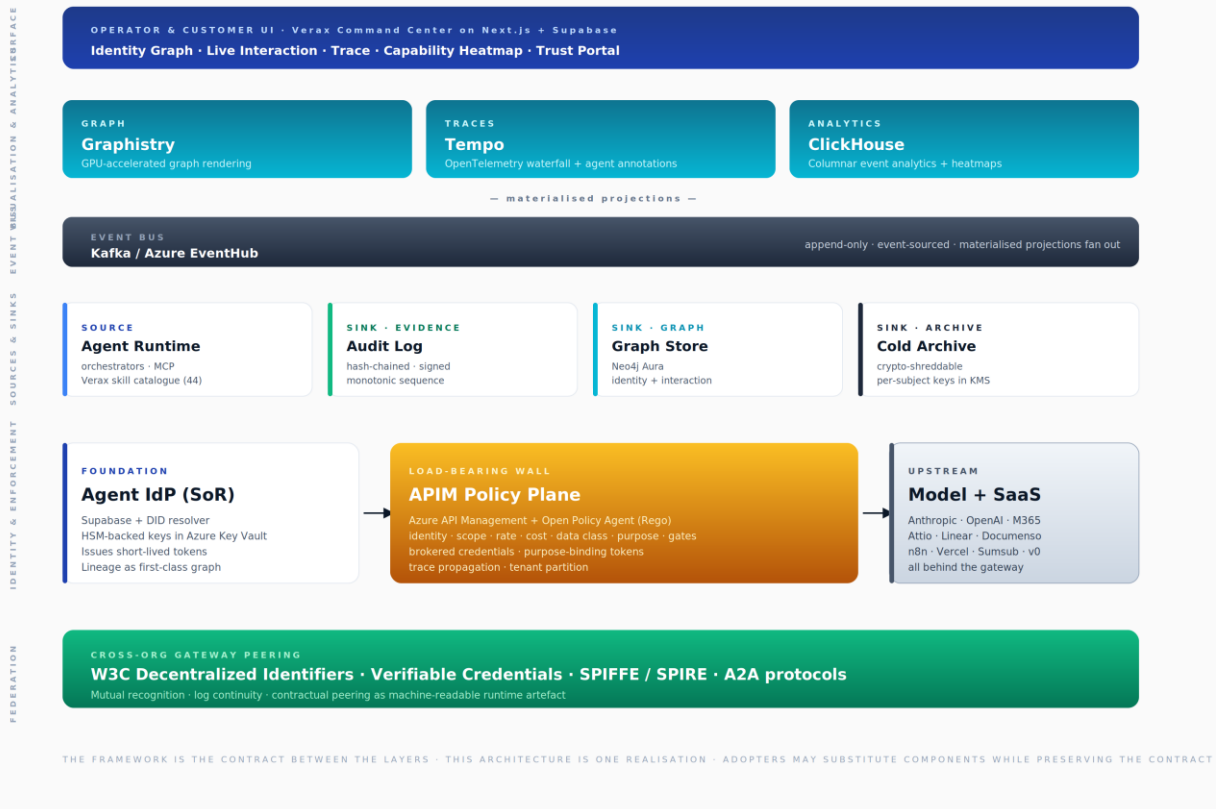
The agent identity provider sits beneath the interface, implemented on Supabase with a custom DID resolver, and is backed by hardware-security-module-grade keys held in Azure Key Vault. The audit log is hash-chained, signed at source by the policy enforcement plane, and held on a hot-path event bus (Kafka) feeding ClickHouse for analytics; cold storage uses S3 or Azure Blob with per-subject KMS keys to support crypto-shredding. The identity and interaction graphs are stored in Neo4j Aura for the production deployment, with Memgraph evaluated for streaming workloads where lower-latency graph mutations are needed. Visualisation uses Graphistry embedded in the Command Center, with OpenTelemetry traces flowing into Tempo for the trajectory view.

The policy enforcement plane uses Azure API Management with an Open Policy Agent sidecar; rulebooks are compiled to Rego for execution in the OPA runtime. Trace context follows the W3C Trace Context standard, with the framework's custom agent-lineage header carrying the multi-hop call tree. Skills are drawn from the existing Verax catalogue (currently approximately forty-four skills), each versioned and signed under the framework. Tooling integrations, Microsoft Graph, Azure management plane, Documenso, Linear, Attio, n8n, Vercel, KYC providers, cost-tracking services, design tooling, are all wrapped behind the policy enforcement plane with brokered credentials, so that the agent never sees the underlying API keys for any third-party service.

FIGURE 8 · REFERENCE ARCHITECTURE

The Verax implementation.

One realisation of the framework — not the only one. The contract is what matters; this is one way to honour it.



13.2 Developer experience , the golden path

If creating a compliant agent is harder than creating a non-compliant one, developers route around the framework. The authoring experience must therefore make the right thing the easy thing. The Verax reference implementation provides a small set of commands that, between them, scaffold a compliant agent end-to-end.

- **verax agent new.** Scaffolds a profile with sane defaults: residency, risk class, baseline rulebooks, capability stub. The new agent is registered in the system of record and emitted to the audit log before any runtime work begins.
- **verax skill bind.** Binds a versioned, signed skill from the catalogue. The binding is content-addressable and recorded in the system of record alongside the agent.
- **verax agent attest.** Runs the local conformance suite, capability lint, rulebook compatibility, ethics-review prerequisites, and produces an attestation request. Attestation is the gate that admits the agent to the production fleet.
- **verax agent simulate.** Runs the agent against a battery of red-team probes locally before deployment. Findings either resolve through code changes or are explicitly accepted as residual risk.

- **verax agent deploy.** Cannot succeed without the Agent Bill of Materials, the attestation, the residency setting and the rulebook bindings all present and valid.

Conformance is checked at build time, not at runtime. The continuous-integration gate is the framework's most effective single enforcement mechanism: an agent that does not pass build-time conformance never reaches an environment where runtime conformance can be tested.

13.3 Adoption staging , incremental, not big-bang

A team can adopt the framework in the following order, deriving value at each step. First, issue identities for existing agents and generate Agent Bills of Materials retroactively, approximately one week. Second, route egress through the policy enforcement plane with brokered credentials and enable basic audit emission, two to four weeks. Third, bind initial rulebooks for the most regulated capabilities first, prioritising personally-identifiable information handling and external communications, two weeks. Fourth, stand up the audit pipeline with hash chaining and cold archival, two to three weeks. Fifth, light up the visualisation layer, starting with the interaction graph and the trajectory view, with the capability heatmap following shortly after, three to four weeks. Sixth, convene the ethics committee and run reviews on existing high-risk agents, an ongoing concern rather than a discrete project. Seventh, add deterministic replay and crypto-shredding capabilities and bind insurance cover, four to eight weeks. Eighth, open the trust portal to customers, two to three weeks.

Total elapsed time for a small team operating in parallel: approximately one quarter. The framework's experience is that organisations which attempt big-bang adoption miss most of the value of staged adoption, each stage produces operational benefit independently, and incur significantly higher risk of stalling at the last quarter of the work, when the easy wins are exhausted and the harder integration tasks remain.

14. Framework Governance, Open Questions and Call to Action

14.1 Why this needs to be open

A proprietary framework for agent identity is a contradiction. The agent economy is cross-organisational by nature: federation, agent-to-agent operation, customer-facing transparency, regulator interaction, none of these works if the framework is recognised only by parties who buy from the framework's author. A proprietary specification produces vendor lock-in, but it also produces ecosystem failure: counterparties cannot federate with a system whose attestation semantics are private, regulators cannot rely on credentials whose verification is gated, and customers cannot procure with confidence against a moving target. The framework is therefore offered as an open specification, with the explicit intention that it become a community artefact rather than a Verax artefact in the long run.

14.2 Governance model

The specification is offered under the Creative Commons Attribution-ShareAlike 4.0 Licence, versioned through a Request for Comments-style change process, with open contribution. The reference implementations are offered under the Apache 2.0 Licence and hosted in public repositories. Conformance is governed by a community-defined test suite; vendors may self-attest or seek third-party attestation, and a Framework-Conformant mark is granted on passing the suite. Stewardship of versions 0.x rests with Verax; version 1 will be governed by a working group with rotating chair, transparent decision log, and a defined path to a neutral foundation, a CNCF, OpenSSF or Linux Foundation AI and Data sub-project, once adoption justifies the move. The specification is semantically versioned; deprecation occurs slowly and with migration playbooks; backwards compatibility of the audit log is preserved across all versions.

14.3 Open questions

Several questions remain unresolved at version 0.1. They are listed here as explicit invitations for contribution from the community.

- **Agent legal personality.** Do autonomous agents need taxpayer identification numbers, wallets and contractual capacity? The work on agentic payments points in this direction; the framework defers a position to version 1 or later.
- **Cross-jurisdiction discovery.** When US discovery seeks audit logs of agents resident in the EU, what is the practical answer? GDPR Article 48 pulls in one direction; the US Federal Rules of Civil Procedure pull in another. Sector-specific playbooks are needed.
- **Model versioning at scale.** What is the minimum viable Agent Bill of Materials for closed-weight foundation models where the provider does not expose the fingerprint or the

training-data summary? The framework's current stance assumes full disclosure; the practical compromise needs definition.

- **Inter-organisation consent.** How does a consent receipt issued by one organisation flow through an agent-to-agent delegation into another organisation's audit without leaking personally-identifiable information? Selective-disclosure techniques are part of the answer; the operational protocol is not yet standardised.
- **Long-tail blessing portability.** Can a blessing earned in one organisation be relied upon in another without re-attestation? When yes, when no, and on what evidence?
- **Sustainability.** Per-agent carbon attribution is increasingly material to procurement and to regulatory reporting. Should it be a required field of the Agent Bill of Materials, and at what granularity?

14.4 Call to action

To operators running agents in production today: pick one section of this paper and adopt it. Identity first if you have nothing; the policy enforcement plane second if you already have identity; audit third if you have both. Each step is independently valuable, and the framework is designed to be sequenced rather than swallowed whole.

To standards bodies, the W3C Verifiable Credentials Working Group, the IETF, ISO/IEC JTC 1, NIST, and the EU AI Act implementation teams, the framework names where existing standards apply directly and where gaps remain. Those gaps are the work, and the framework's authors invite engagement on them.

To regulators: this framework is a deliberate offering of evidentiary structure that satisfies both your obligations and ours. Formal review is invited, and the framework's authors are available to walk through specific aspects with supervisory teams.

To insurers and underwriters: the framework's audit, replay and crypto-shred properties are designed to be the evidence base needed to bind cover with confidence. Collaboration on policy templates and on premium-reduction criteria is invited.

To customers: ask your vendors which stage of the maturity ladder in section 11.10 they operate at. Make stage three the procurement baseline for any agent that touches your data. Make stage four the procurement baseline for any agent that operates in your regulated work.

To contributors: the specification, the reference implementations, the conformance suite and the rulebook library will be hosted in public repositories. Pull requests and issues are how this becomes a movement rather than a paper. The framework will be useful to the extent it is adopted, contributed to and improved by the community.

Appendix A, Glossary

Term	Definition
Agent	A software actor with stable identity, declared purpose, bounded capabilities, the ability to call tools and other agents, and a recorded behavioural history.
ABOM	Agent Bill of Materials, signed manifest of the model, prompt, tools, skills, dependencies and rulebooks that compose a specific agent.
Blessing	Earned credential evidencing demonstrated competence; signed as a Verifiable Credential; expirable; revocable.
Capability	A single, narrow permission such as <code>cap.attio.people.read</code> . Composable, scope-bound.
DID	W3C Decentralized Identifier, a portable, resolvable identifier independent of any single registry.
Lineage	Parent, child, mentor and delegator relationships among agents; queryable as a first-class graph.
Mode	Per-action classification: human-led, human-intervened or autonomous.
Rulebook	A codified, versioned, attestable statement of operational rules with enforcement-layer binding and evidence-event binding.
SoR	System of Record, the master agent identity provider.
VC	W3C Verifiable Credential, a signed, selectively-disclosed claim.

Appendix B, Standards Referenced

The framework references and builds on the following standards: W3C DID Core 1.0; W3C Verifiable Credentials 2.0; W3C Trace Context; OAuth 2.1; OpenID Connect; SPIFFE / SPIRE; OpenTelemetry; RFC 8392 (CWT); RFC 9162 (Certificate Transparency log structure); ISO/IEC 27001; ISO/IEC 27701; ISO/IEC 42001; the NIST AI Risk Management Framework 1.0; the EU AI Act (Regulation 2024/1689); the General Data Protection Regulation; the UK Data Protection Act 2018; HIPAA; PCI-DSS; the US Federal Reserve's SR 11-7 model risk management guidance; the Financial Conduct Authority's SYSC handbook; the Model Context Protocol; and OpenAPI 3.1.

Appendix C, Versioning

Version	Date	Status	Notes
0.1	April 2026	Draft	Initial public draft. Open for review and contribution. Minor textual corrections to the original April 2026 publication: typographic fixes, refinement of authorship attribution and tightening of selected jurisdictional and standards references.

This framework is offered openly. It will be useful to the extent it is adopted, contributed to and improved by the community. Pull requests welcome.

PROOF CERTIFICATE

Veracityos · issued via VeraSign

This document's signing history is sealed and independently anchored.

Every event below is a leaf in a tamper-evident hash chain. The chain's Merkle root was committed to the external trust roots listed under Anchors. Anyone can verify this certificate's claims without contacting Veracityos or VeraSign — see How to verify.

DOCUMENT

Name	Agent Identity Framework Whitepaper - Published.pdf
Subject	Agent Identity Framework Whitepaper Published
Envelope ID	env_0mrt4mpmt6ea99f3c4f6aae22
Status	sealed
SHA-256 of document	sha256:a7d51c26686bee068339e5470c7b5e5d31b39383867b09c4ca0df7cc86af183a
Chain lineage value	sha256:f652bafae7d52c5c4d4c2701d2b8e6aaf50a1585f4a58246154a7c28d6bea64b

The lineage value is an internal chain hash carried forward by the sealer. It is not the hash of any file and is not something you can or should reproduce from your copy — compare your document against “SHA-256 of document” above.

PARTIES (2)

Anthony Penwright <ap@veracityos.io>

signer · identity: email link · signed · signed 2026-07-20 11:34:28 UTC

Anthony <ap@credible.uk.com>

signer · identity: email link · signed · signed 2026-07-20 11:35:55 UTC

COMPLETED FIELDS (5)

What each party entered. These values are hashed into the chain at the moment they were submitted (see the field_completed events below), so they cannot be altered after the fact any more than the signatures can.

Anthony Penwright

Date signed (p2)	2026-07-20
attachment (p2)	Passport Photo.jpeg · sha256:ef9d7fe97bd0888378da29101529860a9702863e6bbb99455cba9292d9b6025a
Signature (p2)	Anthony Penwright

Anthony

Signature (p2)	Anthony
Date signed (p2)	2026-07-20

AUDIT TRAIL (12 EVENTS)

Each event is hash-chained to its predecessor: prev_event_hash binds this list into a sequence that cannot be reordered, inserted into, or edited without breaking every subsequent hash.

01. created 2026-07-20 11:13:51 UTC

sender (console) · console · assurance: low
leaf sha256:6fd5a2a339aa229410d0247089919f06c09610c48c1d795639fca9cf8523a010

02. sent 2026-07-20 11:13:58 UTC

sender (console) · console · assurance: low
leaf sha256:b2484595dc7fc5041f2785449625c66fb019a70c21caec39e1f672828cbb7d66

03. field_completed 2026-07-20 11:17:52 UTC

Anthony Penwright · email_link · assurance: low
leaf sha256:621d1e8b3532089e56f16edcc2ce94e09e6b40c1b63d34942d901e927cfe9581

04. field_completed 2026-07-20 11:34:11 UTC

Anthony Penwright · email_link · assurance: low
leaf sha256:beb6b838c9b738ff738beafe9ef20b1742feaec940fff940cd9ba4c2d03e41f3

05. viewed 2026-07-20 11:34:14 UTC

Anthony Penwright · email_link · assurance: low
leaf sha256:46dda4f7e84a5d4d24177a042fe0b3302cfe3d634d01fe0bc982585cbe6d82e

06. consent_given 2026-07-20 11:34:21 UTC

Anthony Penwright · email_link · assurance: low
leaf sha256:0b36783babe5f66c3341be5f21a3b10e38b0c1556a24a4c883a7fa5780ac7fa3

07. signed 2026-07-20 11:34:29 UTC

Anthony Penwright · email_link · assurance: low
leaf sha256:70731b21ee54669e4ff57dc634921d2cd1386982f1bd7e74c1eee56fcf7400cf

08. field_completed 2026-07-20 11:35:41 UTC

Anthony · email_link · assurance: low
leaf sha256:8782fb6ec1837f8959aec080751f5859d4221cdc8b01e6e654fb103a10694f44

09. viewed 2026-07-20 11:35:44 UTC

Anthony · email_link · assurance: low
leaf sha256:b94940d14c79b7a28817c2c51ee37c0b6526556dced02f7c4422bd02d37d97a5

10. consent_given 2026-07-20 11:35:50 UTC

Anthony · email_link · assurance: low
leaf sha256:2b7d980b772d37c73e9c26c2a0dbea647174efb57c22238c81d0a0c363aa95b6

11. signed 2026-07-20 11:35:56 UTC

Anthony · email_link · assurance: low
leaf sha256:a422022438d01b7258015664d98ad73da05b76251ed1fd0122969c347308ca8e

12. sealed 2026-07-20 11:35:58 UTC

sender (console) · console · assurance: low
leaf sha256:6d16ea6c2130c6078885643cf1b8bd42d5fe999840f0b15630e94cefa57f18e3

ANCHORS

Merkle root sha256:da28171e2d448d6e717dbb321feed84955c9c7238c203a95c03f7d2a5931e9a0

● Commercial timestamp (RFC 3161)

Authority: <https://freetsa.org/tsr> · anchored 2026-07-20 11:36:00 UTC

HOW TO VERIFY — WITHOUT US

This certificate is a set of claims you do not have to take on trust. The sealed package (JSON) contains every event, its Merkle leaf, an inclusion proof against the root above, and the anchor receipts. Verification is arithmetic:

1. Recompute each event's leaf hash and check it matches the leaf printed above.
2. Walk each inclusion proof to confirm every leaf commits to the Merkle root.
3. Check the anchor receipt independently: an RFC 3161 token verifies against the timestamp authority's certificate chain; an OpenTimestamps attestation verifies against the Bitcoin blockchain. Neither requires VeraSign.
4. Hash the document you hold and confirm it equals "SHA-256 of document" above. (Ignore the chain lineage value — it is an internal hash, not the hash of a file.)

Verify online <https://sign.veracityos.io/verify>

Steps 1–4 require only SHA-256 and the anchor's own public standard (RFC 3161 for the timestamp, OpenTimestamps for the Bitcoin attestation). No VeraSign software, account or service is needed to check any claim on this page — that is the point of it.

SCOPE OF THIS CERTIFICATE

This certificate attests to the integrity and timing of the signing history recorded above, and to the identity assurance actually captured for each party (shown per event). It is not a PAdES-embedded digital signature: the document bytes are bound to this chain by their SHA-256, not by a signature embedded inside the PDF. It does not attest to the legal capacity of any party, nor to the truth of the document's contents.

SIGNER ATTACHMENT

Passport Photo.jpeg

